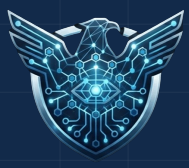
Livro-curso em cadernos modulares

Física estatística e Python aplicados ao mercado

QUANTUM STRATEGIC AI



Econofísica Aplicada ao Mercado Financeiro

Física estatística, sistemas complexos e Python aplicados a dados de mercado

EDIÇÃO EM CONSTRUÇÃO

Caderno 0 · Python para finanças quantitativas
Caderno 1 · Fundamentos da Econofísica

AUTOR

Cristiano Alves

Rio de Janeiro · 2026

Cristiano Alves

Econofísica Aplicada ao Mercado Financeiro

Física estatística, sistemas complexos e Python aplicados a dados de mercado

EDIÇÃO EM CONSTRUÇÃO

Cadernos 0 e 1

Rio de Janeiro
2026

Econofísica Aplicada ao Mercado Financeiro
Edição em construção, com os Cadernos 0 e 1
Livro-curso organizado em cadernos modulares, um para cada módulo do curso. Esta edição reúne os cadernos já escritos; os demais serão acrescentados à medida que ficarem prontos.

© 2026 Cristiano Alves.

Versão de trabalho 0.1, de 10 de outubro de 2026.
Versão de referência da linguagem: Python 3.12. As versões das bibliotecas estão fixadas no arquivo `codigo/requirements.txt` de cada caderno.

Os dados de mercado usados nos exemplos vêm da B3, do Banco Central do Brasil e do Yahoo Finance (pela biblioteca `yfinance`). Cada leitor baixa a própria cópia, como mostram os cadernos, e deve respeitar os termos de uso de cada fonte.

Este material tem finalidade exclusivamente educacional. Nenhum conteúdo, exemplo ou resultado constitui recomendação de investimento. Resultados passados não garantem resultados futuros.

Esta é uma edição em desenvolvimento. Erros e sugestões podem ser comunicados ao autor para incorporação em edições futuras.

Composto em LaTeX. Fontes: Palatino (texto) e Helvetica (títulos).

Apresentação

Este livro ensina a analisar o mercado financeiro com os métodos da Econofísica: as ferramentas da física estatística e dos sistemas complexos aplicadas a dados reais de preços, com Python como instrumento de trabalho. Ele acompanha um curso organizado em onze módulos, do nivelamento em programação ao projeto integrador, e é escrito em cadernos, um para cada módulo.

Cada caderno é um texto completo: tem a sua apresentação, a ficha do módulo correspondente, os seus capítulos, exercícios e referências, e o seu próprio código. Por isso, cada um pode ser estudado separadamente. Reunidos, formam uma trilha contínua, em que cada caderno se apoia nos anteriores.

Esta edição reúne os dois cadernos já escritos. O Caderno 0 prepara o terreno: ambiente de trabalho, Python, NumPy, pandas, gráficos, estatística e a construção de uma base de dados de mercado com cotações da B3, do Banco Central e do Yahoo Finance. O Caderno 1 apresenta a Econofísica: a sua história, de Bachelier a Mandelbrot, a hipótese dos mercados eficientes posta à prova e os fatos estilizados que todo modelo de mercado precisa reproduzir. Os cadernos seguintes serão acrescentados a esta edição à medida que ficarem prontos, conforme o plano da página vi.

O autor
2026

Como usar este livro

A organização. Cada caderno é uma parte do livro e começa com a sua apresentação e com a ficha do módulo, que reproduz a ementa do projeto pedagógico. A numeração dos capítulos, das listagens, das figuras e dos exercícios recomeça em cada caderno: o "Capítulo 3" citado dentro do Caderno 1 é o Capítulo 3 do Caderno 1. O cabeçalho das páginas indica sempre o caderno em que você está.

Os três tempos. Cada capítulo foi pensado para ser lido antes do encontro, praticado durante o encontro e consolidado depois dele, com os exercícios.

As caixas. Quatro caixas reaparecem em todos os cadernos:

Contexto de mercado

Liga o conceito estudado a um fato do mercado financeiro ou a um tema de outro módulo.

Armadilha comum

Aponta um erro frequente, de sintaxe, de lógica ou de método, e mostra como evitá-lo.

Boa prática

Recomendações que separam o código que apenas funciona daquele que se pode reproduzir, conferir e reaproveitar.

Nota

Um esclarecimento, uma definição ou um detalhe que vale a pena guardar.

Os exercícios. Cada capítulo termina com exercícios em três níveis: *essencial* (fixação), *aplicado* (o mesmo método num problema novo) e *desafio* (extensão aberta).

O código e os dados. O código de cada caderno fica na pasta `codigo/` do caderno, com um arquivo por listagem e o arquivo `requirements.txt` com as versões fixadas. As saídas impressas no livro não foram digitadas: são o que o Python imprimiu ao executar as listagens, em sequência, com essas versões. Os dados de mercado não acompanham o livro: cada caderno mostra como baixá-los, e cada leitor guarda as próprias cópias datadas.

Plano do livro

O livro terá onze cadernos, um para cada módulo do curso, num total de 84 horas. A Tabela 1 mostra o plano completo e os cadernos incluídos nesta edição.

Tabela 1 Os cadernos do livro, com a carga horária do módulo correspondente.

Caderno	Título	Horas	Situação
0	Python para finanças quantitativas	12	**nesta edição**
1	Fundamentos da Econofísica	6	**nesta edição**
2	Distribuições de cauda pesada	8	em preparação
3	Processos estocásticos em finanças	8	em preparação
4	Leis de potência e invariância de escala	6	em preparação
5	Fractais, memória longa e multifractalidade	8	em preparação
6	Correlações, matrizes aleatórias e redes	8	em preparação
7	Entropia e informação em séries temporais	6	em preparação
8	Modelos baseados em agentes	8	em preparação
9	Crises, bolhas e eventos extremos	6	em preparação
10	Projeto integrador	8	em preparação
	Total	**84**	

Sumário

Apresentação iv

Como usar este livro v

Plano do livro vi

Caderno 0 – Python para finanças quantitativas 1

Apresentação do Caderno 0 2

Ficha do Módulo 0 3

1 O ambiente de trabalho 4

1.1 Por que Python na Econofísica . . . 4
1.2 Python, ambiente virtual e versões fixadas . . . 5
1.3 JupyterLab e notebooks . . . 7
1.4 Google Colab como alternativa . . . 7
1.5 Um projeto reprodutível . . . 7
Resumo do capítulo . . . 8
Exercícios . . . 8

2 Python essencial 10

2.1 Variáveis, tipos e operações . . . 10
2.2 Textos e f-strings . . . 11
2.3 Listas, tuplas e dicionários . . . 12
2.4 Decisões e repetições . . . 13
2.5 Funções . . . 14
2.6 A biblioteca padrão: matemática, datas e arquivos . . . 15
2.7 Erros e exceções . . . 16
Resumo do capítulo . . . 17
Exercícios . . . 17

3 NumPy: cálculo vetorizado 19

3.1 Arrays e por que evitar laços . . . 19
3.2 Operações, agregações e *broadcasting* . . . 20

3.3 Indexação e máscaras . . . 21
3.4 Números aleatórios com semente: um passeio aleatório . . . 22
Resumo do capítulo . . . 24
Exercícios . . . 24

4 pandas: séries temporais financeiras 25
4.1 Series e DataFrame . . . 25
4.2 Ler e gravar CSV; o índice de datas . . . 26
4.3 Seleção, dados faltantes e dias sem pregão . . . 27
4.4 Reamostragem: de diário a mensal . . . 28
4.5 Janelas móveis: média e volatilidade . . . 29
4.6 Junção de fontes: índice e câmbio . . . 30
Resumo do capítulo . . . 31
Exercícios . . . 31

5 Gráficos e estatística 33
5.1 Anatomia de uma figura . . . 34
5.2 Preços e retornos no tempo . . . 34
5.3 Histogramas e escalas logarítmicas . . . 35
5.4 Estatística descritiva . . . 37
5.5 Distribuições no SciPy . . . 37
5.6 Testes de normalidade . . . 39
Resumo do capítulo . . . 40
Exercícios . . . 40

6 Dados de mercado e retornos 41
6.1 Fontes públicas e o registro de dados . . . 41
6.2 As séries históricas da B3 . . . 42
6.3 Conferindo fontes: preço bruto e preço ajustado . . . 44
6.4 Retornos simples e logarítmicos . . . 46
6.5 Agregação temporal . . . 47
6.6 Prática principal: a base de dados do curso . . . 48
6.7 Lista de verificação de reprodutibilidade . . . 50
Resumo do capítulo . . . 51
Exercícios . . . 51

Referências do Caderno 0 52

Caderno 1 – Fundamentos da Econofísica **55**

Apresentação do Caderno 1 **56**

Ficha do Módulo 1 **57**

1 O que é a Econofísica **58**
1.1 Uma definição . . . 58
1.2 Por que físicos olharam para os mercados . . . 59
1.3 Objetos de estudo e métodos . . . 59
1.4 Econofísica, economia e finanças . . . 60
1.5 Os dados deste caderno . . . 60
Resumo do capítulo . . . 62
Exercícios . . . 62

2 De Bachelier a Mandelbrot **64**
2.1 Bachelier e o passeio aleatório (1900) . . . 64
2.2 Osborne e o movimento browniano geométrico (1959) . . . 65
2.3 Mandelbrot e as caudas pesadas (1963) . . . 66
2.4 A física estatística chega aos mercados (anos 1990) . . . 67
Resumo do capítulo . . . 68
Exercícios . . . 69

3 Mercados eficientes e passeio aleatório **70**
3.1 A hipótese dos mercados eficientes . . . 70
3.2 Do passeio aleatório à ausência de autocorrelação . . . 71
3.3 Testando: correlograma e Ljung-Box . . . 71
3.4 A eficiência muda com o tempo . . . 73
3.5 A razão de variâncias . . . 74
Resumo do capítulo . . . 75
Exercícios . . . 75

4 O modelo gaussiano e os fatos estilizados **76**
4.1 Por que a normal é a referência . . . 76
4.2 Os fatos estilizados . . . 77
4.3 Prática principal: o Ibovespa contra uma amostra gaussiana . . . 78
4.4 Gaussianidade agregacional . . . 80
4.5 Assimetrias: perdas e alavancagem . . . 81
Resumo do capítulo . . . 82
Exercícios . . . 82

5 Críticas e diálogo com a econometria **84**
5.1 As críticas 84
5.2 A resposta da econometria ao agrupamento de volatilidade 85
5.3 Como este curso se posiciona 86
Resumo do capítulo 87
Exercícios 87

Referências do Caderno 1 **88**

CADERNO

0

Python para finanças quantitativas

Módulo 0 do curso · 12 horas

Apresentação do Caderno 0

Os preços do mercado financeiro não se comportam como os livros-texto clássicos supunham. Entre 2016 e 2025, o Ibovespa caiu mais de 10% num único dia quatro vezes, todas em março de 2020; um modelo gaussiano ajustado aos mesmos dados trataria cada uma dessas quedas como praticamente impossível. Períodos agitados e períodos calmos se alternam em vez de se distribuírem ao acaso. A Econofísica nasceu para estudar esses fenômenos com as ferramentas da física estatística e dos sistemas complexos. Este livro-curso leva esses métodos para a prática, com dados reais e com Python como instrumento de trabalho.

O livro é construído em cadernos, um para cada módulo do curso. Cada caderno pode ser estudado sozinho, mas todos seguem a mesma trilha: dos fatos estilizados dos retornos às caudas pesadas, dos processos estocásticos aos fractais, das correlações aos modelos de agentes e ao risco de eventos extremos.

Este é o Caderno 0. Ele não trata ainda de Econofísica: é a base técnica sobre a qual todos os outros se apoiam. Aqui você monta o ambiente de trabalho, aprende o Python necessário, manipula séries de preços com NumPy e pandas, faz gráficos e estatísticas e, por fim, constrói a sua própria base de dados de mercado, com cotações da B3, do Banco Central e do Yahoo Finance. Ao longo do caminho, os primeiros sinais de que os retornos não são gaussianos já aparecem nos seus próprios gráficos.

Escrevi pensando em dois leitores: quem nunca programou e quem já programa em outra linguagem e precisa aprender o jeito Python de trabalhar com dados. Ao primeiro, peço paciência com os Capítulos 1 e 2, que andam rápido. Ao segundo, sugiro ler esses dois capítulos na diagonal e se concentrar nos Capítulos 4 a 6. A ambos, recomendo o mesmo: digitar e executar cada listagem. Programação, como a física experimental, se aprende com as mãos.

O autor
2026

Ficha do Módulo 0

Esta ficha reproduz o Módulo 0 do projeto pedagógico do curso e mostra onde cada item é tratado neste caderno.

Módulo	0 — Python para finanças quantitativas
Eixo	Ferramentas: a base técnica de todos os módulos
Carga horária	12 horas: 4 teóricas e 8 práticas
Pré-requisitos	Cálculo, probabilidade e estatística básicas; noções de mercado (ações, índices, retorno). Programação não é exigida.
Prática principal	Montar uma base de preços diários de ações e índices brasileiros e internacionais, com retornos, estatísticas descritivas e gráficos (Seção 6.6).

Objetivos do módulo. Ao final, o aluno deve ser capaz de:

- configurar um ambiente Python reprodutível, com versões fixadas (Capítulo 1);
- manipular séries de preços com pandas e calcular retornos em diferentes horizontes (Capítulos 4 e 6);
- produzir estatísticas descritivas e gráficos de séries financeiras (Capítulo 5);
- obter dados de fontes públicas, registrando origem e data (Capítulos 4 e 6).

Tabela 1 Itens da ementa do Módulo 0 e capítulos deste caderno.

Item da ementa	Capítulo	Horas
Ambiente: Python, Jupyter, pacotes e ambientes virtuais	1	1,5
Tipos, estruturas de dados, controle de fluxo e funções	2	2,5
NumPy: arrays, operações vetorizadas, números aleatórios com semente	3	1,5
pandas: séries temporais, datas, reamostragem, janelas móveis, junção	4	2,5
Matplotlib e SciPy: gráficos, distribuições e testes	5	2,0
Obtenção de dados; preços e retornos; agregação; reprodutibilidade	6	2,0
Total		**12,0**

CAPÍTULO 1

O ambiente de trabalho

◎ Objetivos do capítulo

- Explicar por que Python é a ferramenta do curso.
- Criar um ambiente virtual com Python 3.12 e instalar as bibliotecas em versões fixadas.
- Trabalhar no JupyterLab e conhecer o Google Colab como alternativa.
- Organizar uma pasta de projeto em que qualquer pessoa reproduza os seus resultados.

1.1 Por que Python na Econofísica

A Econofísica trabalha com séries longas de preços, simulações de milhares de trajetórias, matrizes de correlação com centenas de ativos e ajustes estatísticos que precisam ser repetidos a cada nova amostra. Nada disso cabe numa planilha. É preciso uma linguagem de programação, e o Python reúne três vantagens para esse trabalho.

A primeira é o ecossistema científico. Bibliotecas como NumPy (cálculo com vetores e matrizes), pandas (tabelas e séries temporais), SciPy (estatística e otimização) e Matplotlib (gráficos) são gratuitas, de código aberto e amplamente usadas em pesquisa (HARRIS *et al.*, 2020; VIRTANEN *et al.*, 2020). A segunda é a legibilidade: um código Python bem escrito se lê quase como a descrição do cálculo, o que facilita conferi-lo. A terceira é a reprodutibilidade: com as versões das bibliotecas registradas, o mesmo código produz o mesmo resultado em qualquer computador.

Contexto de mercado

Todos os módulos seguintes do curso usam as ferramentas deste caderno. As caudas pesadas do Módulo 2 são estimadas com NumPy e SciPy; os expoentes de Hurst do Módulo 5, calculados sobre séries do pandas; as redes de correlação do Módulo 6, construídas a partir de matrizes do NumPy. O tempo investido aqui volta em todos os cadernos.

1.2 Python, ambiente virtual e versões fixadas

Um *ambiente virtual* é uma pasta que contém uma cópia isolada do Python e das bibliotecas de um projeto. Instalar uma biblioteca no ambiente virtual não afeta nenhum outro projeto do computador, e vice-versa. É assim que se garante que o código deste caderno rode com as mesmas versões com que foi testado.

1.2.1 Instalar o Python

O caderno usa o Python 3.12, que é também a versão mínima exigida pelas versões de NumPy e SciPy adotadas. Em Linux, ele costuma estar disponível pelo gerenciador de pacotes da distribuição; em Windows e macOS, pelo instalador do site oficial (`https://www.python.org`). No Windows, marque a opção que acrescenta o Python ao *PATH* durante a instalação.

1.2.2 Criar e ativar o ambiente

Na pasta do projeto, crie o ambiente virtual e ative-o. Os comandos mudam conforme o sistema operacional (Listagens 1.1 e 1.2).

Listagem 1.1 Criar e ativar o ambiente virtual no Linux e no macOS.

```
# Linux e macOS: criar e ativar o ambiente virtual na pasta do projeto
python3.12 -m venv .venv
source .venv/bin/activate
python --version
```

Listagem 1.2 Criar e ativar o ambiente virtual no Windows.

```
REM Windows (Prompt de Comando): criar e ativar o ambiente virtual
py -3.12 -m venv .venv
.venv\Scripts\activate
python --version
```

Com o ambiente ativado, o nome `(.venv)` aparece no início da linha do terminal, e os comandos `python` e `pip` passam a se referir ao Python do ambiente.

1.2.3 Instalar as bibliotecas em versões fixadas

O arquivo `requirements.txt` lista as bibliotecas do caderno e as suas versões exatas (Listagem 1.3). Essas versões foram instaladas e testadas em conjunto antes da publicação deste caderno.

Listagem 1.3 O arquivo de requisitos do Caderno 0.

```
# Caderno 0: versões fixadas, testadas em conjunto em 10/10/2026
# Python 3.12 ou superior
numpy==2.5.3
pandas==3.0.6
scipy==1.18.1
matplotlib==3.11.2
yfinance==1.7.0
python-bcb==0.4.0
```

```
jupyterlab==4.6.4
```

O comando da Listagem 1.4 instala exatamente essas versões e abre o JupyterLab.

Listagem 1.4 Instalar as versões fixadas e abrir o JupyterLab.

```
# Com o ambiente ativado: instalar as versões fixadas e abrir o JupyterLab
pip install -r requirements.txt
jupyter lab
```

Boa prática

Fixe as versões com ==, e não com >=. Uma versão nova de uma biblioteca pode mudar o formato de uma saída, renomear uma função ou alterar um valor padrão. Com as versões fixadas, o código que funcionou hoje continua funcionando daqui a um ano. Quando for atualizar, atualize de propósito: mude as versões, execute tudo de novo e confira os resultados.

1.2.4 Verificar o ambiente

A primeira listagem em Python do caderno confere se o ambiente está correto: imprime a versão do Python e a de cada biblioteca.

Listagem 1.5 Verificar as versões instaladas.

```
import sys
from importlib.metadata import version

print("Python     ", sys.version.split()[0])
for pacote in ["numpy", "pandas", "scipy", "matplotlib", "yfinance",
               "python-bcb"]:
    print(f"{pacote:<11}", version(pacote))
```

```
Python      3.12.3
numpy       2.5.3
pandas      3.0.6
scipy       1.18.1
matplotlib  3.11.2
yfinance    1.7.0
python-bcb  0.4.0
```

Se alguma versão for diferente, o ambiente não foi ativado ou a instalação não terminou. Repita os passos das Listagens 1.1 a 1.4.

Armadilha comum

Instalar bibliotecas sem ativar o ambiente virtual. O `pip` instala então no Python do sistema, e o projeto passa a depender de uma configuração que só existe no seu computador. Antes de qualquer `pip install`, confira se `(.venv)` aparece no terminal.

1.3 JupyterLab e notebooks

O JupyterLab é um ambiente de trabalho no navegador. O seu documento principal é o *notebook* (arquivo com extensão `.ipynb`), que alterna *células de código* e *células de texto*. Cada célula de código é executada com `Shift+Enter`, e a saída aparece logo abaixo dela. Essa alternância entre código, resultado e comentário é ideal para a análise exploratória de dados que você vai fazer em todos os módulos.

O Python que executa as células chama-se *kernel*. Ele guarda na memória tudo o que as células já executadas criaram: variáveis, funções, dados. É por isso que as listagens de um mesmo capítulo deste caderno formam uma sequência: uma listagem usa o que as anteriores criaram, como células de um mesmo notebook.

Armadilha comum

Executar as células fora de ordem. O kernel lembra do que foi executado, e não da ordem em que as células aparecem na página. Um notebook que só funciona porque você executou a célula 7 antes da célula 3 vai falhar na mão de outra pessoa. Antes de entregar um notebook, use *Kernel* → *Restart Kernel and Run All Cells* e confira se tudo roda do início ao fim.

1.4 Google Colab como alternativa

O Google Colab oferece notebooks no navegador, sem instalar nada. É uma boa alternativa para começar, mas tem duas diferenças importantes. Primeiro, as versões do Python e das bibliotecas são escolhidas pelo Google e mudam com o tempo. Segundo, os arquivos criados numa sessão são apagados quando ela termina, a menos que você os salve no Google Drive.

Para usar o Colab com este caderno, comece cada notebook com a célula da Listagem 1.6: ela mostra a versão do Python, que deve ser a 3.12 ou superior, e instala as bibliotecas nas versões fixadas.

Listagem 1.6 Primeira célula de um notebook no Colab. O ponto de exclamação executa o comando no terminal da máquina do Colab.

```
!python --version
!pip install -q numpy==2.5.3 pandas==3.0.6 scipy==1.18.1 \
    matplotlib==3.11.2 yfinance==1.7.0 python-bcb==0.4.0
```

1.5 Um projeto reprodutível

Reprodutível é o projeto em que outra pessoa, com o mesmo código e os mesmos dados, obtém os mesmos números. Três hábitos garantem isso desde o primeiro dia.

Uma pasta por projeto, sempre com a mesma estrutura. A pasta `codigo/` que acompanha este caderno segue a organização abaixo, e você pode usá-la como modelo para os seus projetos.

```
codigo/
|-- requirements.txt      versões fixadas (Listagem 1.3)
```

```
|-- matplotlibrc          estilo das figuras (Capítulo 5)
|-- executar_listagens.py executa todas as listagens e grava as saídas
|-- dados/                cópias datadas dos dados e dados processados
|-- figuras/              figuras geradas pelo código
|-- cap01/ ... cap06/     um arquivo .py por listagem
```

Dados brutos nunca são editados. O arquivo baixado de uma fonte fica como veio, com a data do download no nome. Toda limpeza é feita por código, e o resultado vai para um arquivo novo. Assim, sempre é possível refazer o caminho dos dados brutos até o resultado final.

Sementes fixas. Toda simulação usa números aleatórios gerados a partir de uma *semente* declarada no código. Com a mesma semente, a mesma simulação produz os mesmos números. O Capítulo 3 mostra como.

Nota

Quem quiser ir além pode controlar as versões do código com Git, que registra cada alteração dos arquivos. Não é exigido neste caderno, mas é o próximo passo natural de um projeto reprodutível.

Resumo do capítulo

- O Python, com NumPy, pandas, SciPy e Matplotlib, é a ferramenta de todos os módulos do curso.
- Um ambiente virtual isola as bibliotecas de cada projeto, e o `requirements.txt` fixa as suas versões.
- O JupyterLab executa notebooks célula a célula; o kernel guarda o que já foi executado, e por isso a ordem importa.
- O Colab dispensa instalação, mas não controla versões nem guarda arquivos.
- Reprodutibilidade: uma pasta por projeto, dados brutos intocados e sementes fixas.

Exercícios

Essencial — fixação

Exercício 1.1 Crie o ambiente virtual do caderno, instale as bibliotecas listadas no `requirements.txt` e execute a Listagem 1.5. As versões impressas coincidem com as do livro?

Exercício 1.2 Abra o JupyterLab, crie um notebook e escreva uma célula de texto com o seu nome e uma célula de código que imprima `2 ** 10`. Depois, reinicie o kernel e execute todas as células.

Aplicado — o mesmo método num problema novo

Exercício 1.3 Crie, fora da pasta do caderno, a pasta de um projeto seu com a estrutura da Seção 1.5, um ambiente virtual próprio e um `requirements.txt` com apenas `numpy` e `pandas` nas versões deste caderno. Instale e verifique.

Exercício 1.4 Num notebook, crie uma variável na célula 1, use-a na célula 2 e apague a célula 1. A célula 2 continua funcionando? E depois de reiniciar o kernel? Explique.

Desafio — extensão aberta

Exercício 1.5 Repita o Exercício 1.1 no Google Colab, usando a Listagem 1.6. Qual é a versão do Python no Colab hoje? Que bibliotecas o Colab já traz instaladas e em que versões? Discuta o que isso significa para a reprodutibilidade.

CAPÍTULO 2

Python essencial

◎ Objetivos do capítulo

- Usar variáveis, tipos numéricos e textos, e formatar valores financeiros.
- Organizar dados em listas, tuplas e dicionários.
- Controlar o fluxo de um programa com decisões e repetições.
- Escrever funções documentadas para calcular retornos.
- Usar a biblioteca padrão para matemática, datas e arquivos, e tratar erros.

Este capítulo apresenta o Python que você vai usar no resto do curso, e só ele. Todos os exemplos trabalham com preços reais: os fechamentos da PETR4, da VALE3 e do ITUB4 nos primeiros pregões de 2025, tirados do arquivo oficial de cotações da B3 que o Capítulo 6 ensina a ler. Execute cada listagem na ordem em que aparece, como células de um mesmo notebook.

2.1 Variáveis, tipos e operações

Uma *variável* é um nome associado a um valor. O sinal = faz a associação, e o Python deduz o *tipo* do valor: `int` para números inteiros, `float` para números com casas decimais, `str` para textos e `bool` para verdadeiro ou falso.

Listagem 2.1 Uma posição em ações: preço, quantidade e valor.

```
# Uma posição em ações
preco = 36.77          # R$ por ação: fechamento da PETR4 em 02/01/2025
quantidade = 100       # número de ações
valor = preco * quantidade

print(valor)
print(type(preco), type(quantidade), type(valor))
```

```
3677.0000000000005
<class 'float'> <class 'int'> <class 'float'>
```

O resultado da primeira linha surpreende: `3677.0000000000005`, e não `3677`. Não é um defeito do Python, e sim da forma como todo computador guarda números com casas decimais.

Armadilha comum

Números `float` são guardados em base 2, com cerca de 16 dígitos significativos. Valores como 0,1 ou 36,77 não têm representação exata em base 2, e pequenos erros de arredondamento aparecem nas contas. Por isso, **nunca compare dois `float` com `==`**: use `math.isclose`, que admite uma pequena tolerância (Listagem 2.2). Para exibir valores, arredonde com `round` ou na formatação.

Listagem 2.2 A aritmética de ponto flutuante e como comparar valores.

```
import math

print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
print(math.isclose(0.1 + 0.2, 0.3))
print(round(valor, 2))
```

```
0.30000000000000004
False
True
3677.0
```

2.2 Textos e f-strings

Textos (`str`) ficam entre aspas. A forma mais prática de montar um texto com valores é a *f-string*: um texto precedido da letra `f`, com expressões entre chaves. Depois de dois-pontos, dentro das chaves, vem o formato: `.2f` para duas casas decimais, `,` para separar milhares e `.2%` para porcentagem.

Listagem 2.3 Formatando preços, valores e retornos.

```
ticker = "PETR4"
retorno = 36.38 / 36.77 - 1        # do 1º para o 2º pregão de 2025

print(f"{ticker}: R$ {preco:.2f} por ação")
print(f"Valor da posição: R$ {valor:,.2f}")
print(f"Retorno no dia: {retorno:.2%}")
print(ticker.lower(), ticker[:4], len(ticker))
```

```
PETR4: R$ 36.77 por ação
Valor da posição: R$ 3,677.00
Retorno no dia: -1.06%
petr4 PETR 5
```

Nota

O Python usa o padrão norte-americano: ponto como separador decimal e vírgula nos milhares (`3,677.00`). Para relatórios em português, basta trocar os separadores no texto final, como pede o Exercício 2.3. Nos cálculos, os números continuam sendo números; só a apresentação muda.

2.3 Listas, tuplas e dicionários

Uma *lista* guarda uma sequência de valores, na ordem. As posições começam em **zero**: o primeiro elemento é `fechamentos[0]`, e índices negativos contam do fim (`-1` é o último). Uma *fatia* `[1:3]` pega as posições 1 e 2: o início entra, o fim não.

Listagem 2.4 Os fechamentos da PETR4 numa lista.

```
# Fechamentos da PETR4 nos cinco primeiros pregões de 2025 (fonte: B3)
fechamentos = [36.77, 36.38, 36.21, 36.98, 36.68]

print(fechamentos[0], fechamentos[-1])     # primeiro e último
print(fechamentos[1:3])                    # fatia: posições 1 e 2
print(len(fechamentos), min(fechamentos), max(fechamentos))

fechamentos.append(36.84)                  # pregão de 09/01/2025
print(fechamentos)
```

```
36.77 36.68
[36.38, 36.21]
5 36.21 36.98
[36.77, 36.38, 36.21, 36.98, 36.68, 36.84]
```

Uma *tupla* é como uma lista que não pode ser alterada; serve para agrupar valores que andam juntos, como a data e o preço de um pregão. Um *dicionário* associa *chaves* a *valores*: é a estrutura natural para uma carteira, em que cada ticker tem uma quantidade.

Listagem 2.5 Uma tupla, uma carteira e o valor da carteira.

```
# Uma tupla agrupa valores fixos; um dicionário associa chaves a valores
pregao = ("2025-01-02", 36.77)
data, fechamento = pregao                  # desempacotamento
print(data, fechamento)

carteira = {"PETR4": 100, "VALE3": 50, "ITUB4": 80}        # quantidades
precos = {"PETR4": 36.77, "VALE3": 54.25, "ITUB4": 30.57}  # 02/01/2025

total = 0.0
for ticker, qtd in carteira.items():
    total += qtd * precos[ticker]
print(f"Valor da carteira: R$ {total:,.2f}")
print(carteira.get("BBDC4", 0))            # chave ausente: valor padrão
```

```
2025-01-02 36.77
Valor da carteira: R$ 8,835.10
0
```

O método `.items()` percorre os pares chave-valor do dicionário, e `.get` devolve um valor padrão quando a chave não existe, em vez de interromper o programa com um erro.

2.4 Decisões e repetições

O laço `for` repete um bloco de código para cada elemento de uma sequência. O bloco é definido pela **indentação**: as linhas recuadas em quatro espaços. A Listagem 2.6 calcula o *retorno simples* de cada pregão,

$$r_t = \frac{P_t}{P_{t-1}} - 1,$$

em que P_t é o preço de fechamento no pregão t.

Listagem 2.6 Retorno simples de cada pregão com um laço `for`.

```
# Retorno simples de cada pregão: r = P(t) / P(t-1) - 1
retornos = []
for i in range(1, len(fechamentos)):
    r = fechamentos[i] / fechamentos[i - 1] - 1
    retornos.append(r)

for r in retornos:
    print(f"{r:+.2%}")
```

```
-1.06%
-0.47%
+2.13%
-0.81%
+0.44%
```

As decisões usam `if`, `elif` (“senão, se”) e `else`. O laço `while` repete enquanto uma condição for verdadeira, e é útil quando não se sabe de antemão quantas repetições serão necessárias.

Listagem 2.7 Contando altas e quedas; um laço `while`.

```
altas = quedas = 0
for r in retornos:
    if r > 0:
        altas += 1
    elif r < 0:
        quedas += 1
print(f"{altas} altas e {quedas} quedas")

# while: em que pregão o preço voltou ao primeiro fechamento?
i = 1
while fechamentos[i] < fechamentos[0]:
    i += 1
print(f"Voltou a R$ {fechamentos[0]} ou mais no pregão de índice {i}")
```

```
2 altas e 3 quedas
Voltou a R$ 36.77 ou mais no pregão de índice 3
```

Armadilha comum

Um `while` cuja condição nunca fica falsa repete para sempre. Na Listagem 2.7, se o preço nunca voltasse ao primeiro fechamento, o índice `i` passaria do fim da lista e o programa terminaria com um erro. Sempre pergunte: o que garante que este laço termina?

Muitas vezes um laço só serve para montar uma lista nova. A *compreensão de lista* faz isso numa expressão só, e ainda permite filtrar elementos com `if`.

Listagem 2.8 Compreensões de lista: retornos e só as quedas.

```
# Compreensão de lista: o mesmo cálculo da Listagem 2.6 em uma expressão
retornos = [fechamentos[i] / fechamentos[i - 1] - 1
            for i in range(1, len(fechamentos))]
quedas = [r for r in retornos if r < 0]
print(len(retornos), len(quedas))
print([round(r, 4) for r in retornos])
```

```
5 3
[-0.0106, -0.0047, 0.0213, -0.0081, 0.0044]
```

2.5 Funções

Uma *função* dá nome a um cálculo, para que ele seja escrito uma vez e usado muitas. A definição começa com `def`; as *anotações de tipo* (`p0: float, -> float`) dizem o que a função recebe e devolve; e a *docstring*, entre aspas triplas, explica o que ela faz.

O *retorno acumulado* de um período compõe os retornos de cada pregão:

$$R = (1 + r_1)(1 + r_2) \cdots (1 + r_n) - 1 = \frac{P_n}{P_0} - 1.$$

A Listagem 2.9 implementa as duas formas e usa `assert` para conferir que elas coincidem.

Listagem 2.9 Funções para o retorno simples e o retorno acumulado.

```
def retorno_simples(p0: float, p1: float) -> float:
    """Retorno simples entre dois preços: p1 / p0 - 1."""
    return p1 / p0 - 1


def retorno_acumulado(retornos: list[float]) -> float:
    """Compõe retornos simples: (1 + r1)(1 + r2)...(1 + rn) - 1."""
    fator = 1.0
    for r in retornos:
        fator *= 1 + r
    return fator - 1


print(f"{retorno_simples(36.77, 36.84):.4%}")
print(f"{retorno_acumulado(retornos):.4%}")
assert math.isclose(retorno_acumulado(retornos),
                    retorno_simples(fechamentos[0], fechamentos[-1]))
print("Conferido: compor os retornos diários dá o retorno do período.")
```

```
0.1904%
0.1904%
Conferido: compor os retornos diários dá o retorno do período.
```

Boa prática

Teste as suas funções com casos cujo resultado você conhece. O `assert` interrompe o programa se a condição for falsa: se uma alteração futura quebrar a função, você descobre na hora, e não depois de publicar um resultado errado.

2.6 A biblioteca padrão: matemática, datas e arquivos

O Python vem com uma *biblioteca padrão* extensa. Três dos seus módulos aparecem o tempo todo em finanças.

`math`. Funções matemáticas, como o logaritmo natural. O *retorno logarítmico* entre dois preços é

$$\ell = \ln \frac{P_1}{P_0} = \ln(1 + r).$$

Para variações pequenas, $\ln(1+r) \approx r$, e os dois retornos quase coincidem; para variações grandes, não (Listagem 2.10). O Capítulo 6 mostra por que o retorno logarítmico é o preferido na Econofísica.

Listagem 2.10 Retorno simples e retorno logarítmico.

```
# Retorno logarítmico: ln(p1 / p0). Para variações pequenas, fica muito
# perto do retorno simples; para variações grandes, não.
for p1 in [36.84, 40.00, 50.00]:
    simples = retorno_simples(36.77, p1)
    logaritmico = math.log(p1 / 36.77)
    print(f"R$ {p1:5.2f}   simples {simples:+.4f}   log {logaritmico:+.4f}")
```

```
R$ 36.84   simples +0.0019   log +0.0019
R$ 40.00   simples +0.0878   log +0.0842
R$ 50.00   simples +0.3598   log +0.3073
```

`datetime`. Datas e intervalos de tempo. A diferença entre duas datas dá os dias corridos; `timedelta` soma dias; `strftime` formata a data no padrão brasileiro.

Listagem 2.11 Datas com o módulo `datetime`.

```
from datetime import date, timedelta

inicio = date(2025, 1, 2)
fim = date(2025, 12, 30)
print((fim - inicio).days, "dias corridos")
print(inicio + timedelta(days=30))
print(inicio.strftime("%d/%m/%Y"), inicio.isoformat())
print(date.fromisoformat("2025-06-06").weekday())   # 0 = segunda-feira
```

```
362 dias corridos
2025-02-01
02/01/2025 2025-01-02
4
```

Repare: entre o primeiro e o último pregão de 2025 há 362 dias corridos, mas o arquivo da B3 traz 250 pregões. Fins de semana e feriados não têm negociação. Essa diferença entre *dias corridos* e *dias úteis* vai reaparecer quando você anualizar retornos e volatilidades.

`pathlib`. Caminhos de pastas e arquivos, escritos de um jeito que funciona em qualquer sistema operacional. O operador `/` junta partes do caminho.

Listagem 2.12 Criar uma pasta e gravar e ler um arquivo de texto.

```
from pathlib import Path

pasta = Path("dados") / "exemplo"
pasta.mkdir(parents=True, exist_ok=True)      # não reclama se já existir

arquivo = pasta / "fechamentos.txt"
arquivo.write_text("\n".join(str(p) for p in fechamentos), encoding="utf-8")
print(arquivo, arquivo.exists(), arquivo.stat().st_size, "bytes")
print(arquivo.read_text(encoding="utf-8").splitlines()[:3])
```

```
dados/exemplo/fechamentos.txt True 35 bytes
['36.77', '36.38', '36.21']
```

2.7 Erros e exceções

Quando algo dá errado, o Python interrompe o programa com uma *exceção*, que tem um tipo e uma mensagem. Um caso clássico com dados brasileiros: planilhas e arquivos exportados em português usam vírgula decimal, e o Python não converte esse texto em número. A listagem abaixo termina com erro de propósito; a saída mostra a última linha da mensagem.

Listagem 2.13 Um erro de propósito: converter um número com vírgula decimal.

```
float("36,77")      # vírgula decimal, como nas planilhas brasileiras
```

```
ValueError: could not convert string to float: '36,77'
```

O bloco `try/except` captura a exceção e decide o que fazer. A função da Listagem 2.14 troca a vírgula por ponto e, se mesmo assim não conseguir converter, devolve `None`, o valor que o Python usa para "nada".

Listagem 2.14 Tratando a exceção: conversão tolerante a vírgulas.

```
def para_numero(texto: str) -> float | None:
    """Converte '36,77' ou '36.77' em número; devolve None se não der."""
    try:
        return float(texto.replace(",", "."))
    except ValueError:
        return None


for texto in ["36,77", "36.38", "n/d", ""]:
    print(repr(texto), "->", para_numero(texto))
```

```
'36,77' -> 36.77
'36.38' -> 36.38
'n/d' -> None
```

```
'' -> None
```

Armadilha comum

Capturar exceções demais. Um `except:` sem tipo esconde qualquer erro, inclusive os que indicam um defeito no seu código. Capture só o tipo que você espera (`ValueError`, aqui) e deixe os outros aparecerem.

Resumo do capítulo

- Tipos básicos: `int`, `float`, `str` e `bool`. Valores `float` têm erros de arredondamento: compare com `math.isclose`.
- f-strings formatam valores: `{valor:,.2f}`, `{r:.2%}`.
- Listas (posições a partir de zero), tuplas (valores fixos) e dicionários (chave e valor).
- `for`, `while`, `if/elif/else` e compreensões de lista; o bloco é definido pela indentação.
- Funções com anotações de tipo e docstring; `assert` para conferir resultados.
- `math`, `datetime` e `pathlib`; `try/except` para tratar erros esperados.

Exercícios

Essencial — fixação

Exercício 2.1 Altere a Listagem 2.5 para incluir na carteira 30 ações da BBDC4 a R$ 12,50 e recalcule o valor total.

Exercício 2.2 Modifique a Listagem 2.7 para contar também os pregões em que o retorno, em valor absoluto, passou de 1%.

Exercício 2.3 Escreva uma função `formato_br(valor)` que devolva o texto de um valor no padrão brasileiro, por exemplo `"3.677,00"` para `3677`. Dica: formate com `{valor:,.2f}` e troque os separadores.

Aplicado — o mesmo método num problema novo

Exercício 2.4 Escreva a função `retorno_log_acumulado(precos)` que recebe uma lista de preços e devolve a soma dos retornos logarítmicos de cada pregão. Confira com `assert` que o resultado é igual a $\ln(P_n / P_0)$.

Exercício 2.5 A função `para_numero` da Listagem 2.14 erra com `"1.234,56"`. Por quê? Corrija-a para aceitar o padrão brasileiro com separador de milhares, sem quebrar os casos que já funcionavam.

Desafio — extensão aberta

Exercício 2.6 Usando só listas e laços, escreva uma função que receba uma lista de preços e devolva a maior queda acumulada a partir de um pico (o *drawdown* máximo). Teste com os fechamentos da Listagem 2.4. Você vai reencontrar essa medida no Módulo 9.

CAPÍTULO 3

NumPy: cálculo vetorizado

◎ Objetivos do capítulo

- Criar arrays e calcular retornos sem laços (vetorização).
- Agregar por linhas e colunas com o argumento `axis` e usar *broadcasting*.
- Selecionar dados com máscaras booleanas.
- Gerar números aleatórios reprodutíveis e simular um passeio aleatório de preços.

O NumPy é a base de quase todo o Python científico (HARRIS *et al.*, 2020). O seu objeto central é o *array*: uma tabela de números, todos do mesmo tipo, guardados de forma contígua na memória. Isso permite aplicar uma operação a milhões de valores de uma vez, com código compilado, em vez de percorrê-los um a um num laço Python.

3.1 Arrays e por que evitar laços

Um array é criado a partir de uma lista com `np.array`. Os atributos `dtype`, `shape` e `ndim` informam o tipo dos elementos, o formato e o número de dimensões.

Listagem 3.1 Um array com os fechamentos da PETR4.

```
import numpy as np

fechamentos = np.array([36.77, 36.38, 36.21, 36.98, 36.68, 36.84])
print(fechamentos)
print(fechamentos.dtype, fechamentos.shape, fechamentos.ndim)
```

```
[36.77 36.38 36.21 36.98 36.68 36.84]
float64 (6,) 1
```

Com um array, o retorno de todos os pregões sai numa linha. A fatia `fechamentos[1:]` contém do segundo preço em diante; `fechamentos[:-1]`, do primeiro ao penúltimo. Dividir uma pela outra divide cada preço pelo anterior. Essa forma de escrever operações sobre arrays inteiros, sem laço, chama-se *vetorização*.

Listagem 3.2 Retornos simples e logarítmicos sem laço.

```
# Retornos sem laço: cada fechamento dividido pelo anterior
retornos = fechamentos[1:] / fechamentos[:-1] - 1
print(retornos.round(4))
```

```

log_retornos = np.diff(np.log(fechamentos))
print(log_retornos.round(4))
```

```
[-0.0106 -0.0047  0.0213 -0.0081  0.0044]
[-0.0107 -0.0047  0.021  -0.0081  0.0044]
```

Compare com a Listagem 2.6: o resultado é o mesmo, com uma linha em vez de quatro. E a diferença não é só de estilo. A Listagem 3.3 mede o tempo das duas formas com um milhão de preços.

Listagem 3.3 Laço contra vetorização com um milhão de preços.

```
import time

grande = np.random.default_rng(1).uniform(10, 50, size=1_000_000)

t0 = time.perf_counter()
r_laco = [grande[i] / grande[i - 1] - 1 for i in range(1, len(grande))]
t1 = time.perf_counter()
r_vet = grande[1:] / grande[:-1] - 1
t2 = time.perf_counter()

print(f"laço: {t1 - t0:.3f} s   vetorizado: {t2 - t1:.4f} s")
print(f"vetorizado: cerca de {(t1 - t0) / (t2 - t1):.0f} vezes mais rápido")
print(np.allclose(r_laco, r_vet))
```

```
laço: 0.212 s   vetorizado: 0.0042 s
vetorizado: cerca de 50 vezes mais rápido
True
```

Nota

Os tempos variam de um computador para outro e até entre duas execuções no mesmo computador; a proporção entre eles, de dezenas de vezes, é o que importa. A última linha usa `np.allclose` para confirmar que os dois métodos dão o mesmo resultado, a menos de arredondamentos.

3.2 Operações, agregações e *broadcasting*

Arrays podem ter duas dimensões, como uma planilha: aqui, uma linha por pregão e uma coluna por ação. As funções de agregação (`mean`, `std`, `max`, `sum`) recebem o argumento `axis`, que diz em que direção agregar.

Listagem 3.4 Fechamentos de três ações e agregações por eixo.

```
# Fechamentos na B3 de PETR4, VALE3 e ITUB4 nos 6 primeiros pregões de
# 2025: uma linha por pregão, uma coluna por ação
precos = np.array([
    [36.77, 54.25, 30.57],
    [36.38, 53.24, 29.82],
    [36.21, 52.56, 31.16],
    [36.98, 52.05, 31.50],
    [36.68, 51.55, 30.99],
    [36.84, 51.23, 31.07],
```

```
])
print(precos.shape)
print(precos.mean(axis=0).round(2))    # média de cada ação (desce as linhas)
print(precos.max(axis=1))              # maior preço de cada pregão
```

```
(6, 3)
[36.64 52.48 30.85]
[54.25 53.24 52.56 52.05 51.55 51.23]
```

Armadilha comum

Confundir os eixos. `axis=0` percorre as **linhas** e devolve um valor por **coluna** (a média de cada ação); `axis=1` percorre as colunas e devolve um valor por linha (o maior preço de cada pregão). Uma regra prática: o eixo informado é o que desaparece no resultado.

Quando se opera um array 6×3 com um array de 3 elementos, o NumPy repete o menor ao longo das linhas, sem copiar dados. Esse mecanismo, o *broadcasting*, permite pôr as três ações na mesma *base 100* numa única expressão: cada preço dividido pelo primeiro preço da sua coluna, vezes 100.

Listagem 3.5 Base 100 e desvio-padrão dos retornos das três ações.

```
# Base 100: cada linha dividida pela primeira (broadcasting)
base100 = 100 * precos / precos[0]
print(base100.round(2))

retornos = precos[1:] / precos[:-1] - 1      # as três ações de uma vez
print(retornos.std(axis=0, ddof=1).round(4))
```

```
[[100.   100.   100.  ]
 [ 98.94  98.14  97.55]
 [ 98.48  96.88 101.93]
 [100.57  95.94 103.04]
 [ 99.76  95.02 101.37]
 [100.19  94.43 101.64]]
[0.0129 0.0047 0.0271]
```

O argumento `ddof=1` faz o desvio-padrão dividir por $n - 1$, e não por n: é o desvio-padrão *amostral*, o mesmo que o pandas usa por padrão.

3.3 Indexação e máscaras

Uma comparação aplicada a um array produz um array de `True` e `False`, a *máscara*. Usada como índice, a máscara seleciona só os elementos em que vale `True`. Somar uma máscara conta os `True`.

Listagem 3.6 Máscaras: quedas, seleção e posição do mínimo.

```
quedas = retornos < 0
print(quedas)
print(quedas.sum(axis=0))                    # número de quedas por ação

vale = retornos[:, 1]
print(vale[vale < -0.01].round(4))           # quedas de mais de 1% da VALE3
print(np.where(retornos[:, 2] > 0, "alta", "queda"))    # ITUB4
```

```
print(retornos.argmin(axis=0))                # pior queda de cada uma
```

```
[[ True  True  True]
 [ True  True False]
 [False  True False]
 [ True  True  True]
 [False  True False]]
[3 5 2]
[-0.0186 -0.0128]
['queda' 'alta' 'alta' 'queda' 'alta']
[0 0 0]
```

A função `np.where` escolhe entre dois valores elemento a elemento, e `argmin` devolve a *posição* do menor valor. Nas três ações, a pior queda desses dias foi o primeiro retorno (posição 0), do primeiro para o segundo pregão.

3.4 Números aleatórios com semente: um passeio aleatório

Simular é uma das ferramentas centrais da Econofísica: gerar milhares de trajetórias de preço com um modelo e comparar as suas propriedades com as dos dados reais. Para isso, o NumPy oferece geradores de números aleatórios. Na verdade, são *pseudoaleatórios*: uma sequência determinística que parece aleatória e que é totalmente definida por um número inicial, a *semente*.

Listagem 3.7 Mesma semente, mesmos números.

```
rng = np.random.default_rng(seed=42)
print(rng.normal(0, 0.01, size=3).round(5))

rng = np.random.default_rng(seed=42)          # mesma semente...
print(rng.normal(0, 0.01, size=3).round(5))   # ...mesmos números
```

```
[ 0.00305 -0.0104   0.0075 ]
[ 0.00305 -0.0104   0.0075 ]
```

Boa prática

Crie um gerador com `np.random.default_rng(seed=...)` e use-o em todas as simulações. Evite as funções antigas do tipo `np.random.seed` e `np.random.normal`, que usam um estado global: qualquer outra parte do código que sorteie números muda os seus resultados sem que você perceba.

O modelo mais simples de preço é o *passeio aleatório geométrico*: os retornos logarítmicos diários são sorteados de uma distribuição normal, independentes entre si, e o preço é a exponencial da soma acumulada,

$$P_t = P_0 \exp\Big(\sum_{s=1}^{t} \ell_s\Big), \qquad \ell_s \sim \mathcal{N}(\mu, \sigma^2).$$

A Listagem 3.8 simula cinco trajetórias de 250 pregões, com $\mu = 0{,}03\%$ e $\sigma = 1{,}5\%$ ao dia, valores próximos dos do Ibovespa (Capítulo 5). A Figura 3.1 mostra o resultado.

Listagem 3.8 Cinco passeios aleatórios geométricos.

```python
import matplotlib.pyplot as plt

# Passeio aleatório geométrico: log-retornos normais e independentes
rng = np.random.default_rng(seed=42)
dias, trajetorias = 250, 5
log_r = rng.normal(0.0003, 0.015, size=(dias, trajetorias))
simulados = 100 * np.exp(np.cumsum(log_r, axis=0))

print(simulados.shape)
print(simulados[-1].round(2))            # preço final de cada trajetória

fig, ax = plt.subplots()
ax.plot(simulados)
ax.set_title("Cinco passeios aleatórios a partir de R$ 100")
ax.set_xlabel("pregão")
ax.set_ylabel("preço (R$)")
fig.savefig("figuras/fig_03_01_passeio.pdf")
```

```
(250, 5)
[101.35 115.67 121.67  93.39  64.12]
```

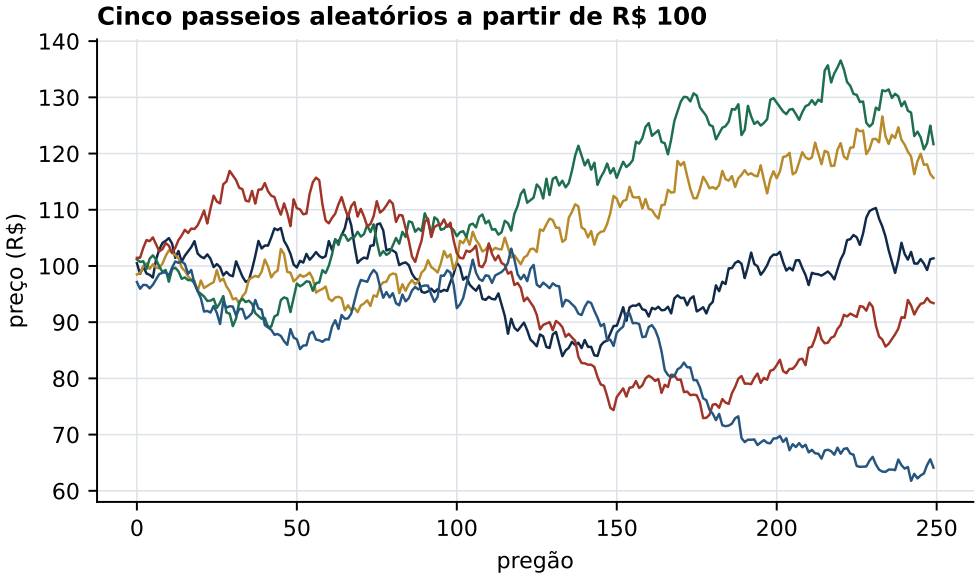


Figura 3.1 Cinco trajetórias de um passeio aleatório geométrico com os mesmos parâmetros. Depois de 250 pregões, os preços finais vão de R$ 64 a R$ 122.

As cinco trajetórias têm exatamente o mesmo modelo e os mesmos parâmetros e, mesmo assim, terminam o ano entre R$ 64 e R$ 122. O acaso sozinho produz tendências que, olhando uma única trajetória, pareceriam ter uma explicação.

Contexto de mercado

O passeio aleatório é a hipótese que Louis Bachelier propôs em 1900 para os preços da Bolsa de Paris (BACHELIER, 1900), e é a referência contra a qual a Econofísica compara os dados reais. Os Módulos 1 e 3 voltam a ele em detalhe. O Capítulo 5 deste caderno já mostra a primeira diferença: nos dados reais, os retornos extremos são muito mais frequentes do que a normal usada aqui prevê.

Resumo do capítulo

- Um array guarda números do mesmo tipo; operações sobre o array inteiro (vetorização) são dezenas de vezes mais rápidas do que laços.
- Retornos sem laço: `p[1:] / p[:-1] - 1` e `np.diff(np.log(p))`.
- `axis=0` agrega ao longo das linhas (um valor por coluna); o eixo informado desaparece no resultado.
- Broadcasting estende o array menor sem copiar dados (base 100 numa expressão).
- Máscaras booleanas selecionam e contam; `np.where` e `argmin` completam a seleção.
- Simulações usam `np.random.default_rng(seed)`: mesma semente, mesmos números.

Exercícios

Essencial — fixação

Exercício 3.1 Usando o array `precos` da Listagem 3.4, calcule o retorno acumulado de cada ação nos seis pregões, sem laço.

Exercício 3.2 Mude a semente da Listagem 3.8 para 7 e execute de novo. O que muda e o que continua igual?

Aplicado — o mesmo método num problema novo

Exercício 3.3 Simule 10 000 trajetórias de 250 pregões com os parâmetros da Listagem 3.8. Em quantas o preço final ficou abaixo de R$ 80? Compare com a probabilidade teórica: o logaritmo do preço final é normal, com média $\ln 100 + 250\mu$ e desvio-padrão $\sigma\sqrt{250}$.

Exercício 3.4 Escreva uma versão vetorizada do drawdown máximo do Exercício 2.6. Dica: `np.maximum.accumulate` calcula o máximo acumulado de um array.

Desafio — extensão aberta

Exercício 3.5 Nas 10 000 trajetórias do Exercício 3.3, conte quantas tiveram pelo menos um dia com queda maior que 5%. Guarde esse número: no Capítulo 5, você vai compará-lo com o que aconteceu no Ibovespa.

CAPÍTULO 4

pandas: séries temporais financeiras

◎ Objetivos do capítulo

- Criar Series e DataFrames indexados por datas.
- Baixar cotações, gravá-las em CSV e lê-las de volta.
- Selecionar períodos, diagnosticar dados faltantes e limpar dias sem pregão.
- Reamostrar séries diárias e calcular estatísticas em janelas móveis.
- Juntar séries de fontes diferentes pela data.

O pandas organiza dados em tabelas com rótulos (McKINNEY, 2010). Para séries financeiras, o rótulo mais importante é a data: com um índice de datas, selecionar um mês, passar de diário para mensal ou alinhar duas fontes vira uma linha de código. Este capítulo usa cotações reais de 2016 a 2025.

4.1 Series e DataFrame

Uma *Series* é uma coluna de valores com um *índice*, que rotula cada valor. Um *DataFrame* é uma tabela de várias Series que compartilham o mesmo índice.

Listagem 4.1 Uma Series e um DataFrame indexados por datas.

```
import pandas as pd

datas = pd.to_datetime(["2025-01-02", "2025-01-03", "2025-01-06"])
petr4 = pd.Series([36.77, 36.38, 36.21], index=datas, name="PETR4")
print(petr4)

tabela = pd.DataFrame({"PETR4": [36.77, 36.38, 36.21],
                       "VALE3": [54.25, 53.24, 52.56]}, index=datas)
print(tabela)
print(tabela["VALE3"].mean())
```

```
2025-01-02    36.77
2025-01-03    36.38
2025-01-06    36.21
Name: PETR4, dtype: float64
            PETR4  VALE3
2025-01-02  36.77  54.25
2025-01-03  36.38  53.24
2025-01-06  36.21  52.56
```

```
53.35
```

4.2 Ler e gravar CSV; o índice de datas

Os dados deste capítulo vêm do Yahoo Finance, pela biblioteca `yfinance`: os fechamentos diários de cinco séries entre 2016 e 2025 (Tabela 4.1). O Capítulo 6 discute as fontes de dados em detalhe; aqui, o objetivo é ter uma tabela real para trabalhar.

Tabela 4.1 As cinco séries usadas no caderno. O sufixo `.SA` indica papéis negociados na B3, e o acento circunflexo, índices.

Código no Yahoo	Série	Moeda
`^BVSP`	Ibovespa, principal índice de ações da B3	pontos
`^GSPC`	S&P 500, índice de ações dos Estados Unidos	pontos
`PETR4.SA`	Petrobras PN	R$
`VALE3.SA`	Vale ON	R$
`ITUB4.SA`	Itaú Unibanco PN	R$

A Listagem 4.2 baixa as cinco séries e grava um arquivo CSV com a **data do download no nome**. Com `auto_adjust=True`, os preços das ações vêm ajustados por proventos e desdobramentos, como convém para calcular retornos (Seção 6.3).

Listagem 4.2 Baixar os fechamentos do Yahoo Finance e gravar uma cópia datada.

```
from datetime import date
from pathlib import Path

import yfinance as yf

ativos = ["^BVSP", "^GSPC", "PETR4.SA", "VALE3.SA", "ITUB4.SA"]
precos = yf.download(ativos, start="2016-01-01", end="2026-01-01",
                     auto_adjust=True, progress=False)["Close"]

arquivo = Path("dados") / f"yahoo_fechamentos_{date.today()}.csv"
precos.to_csv(arquivo)
print(arquivo, precos.shape)
```

```
dados/yahoo_fechamentos_2026-10-10.csv (2581, 5)
```

São 2 581 datas e 5 colunas. A Listagem 4.3 lê o arquivo de volta. O argumento `parse_dates=True` converte a coluna de datas em datas de verdade, e `index_col` a transforma no índice da tabela.

Listagem 4.3 Ler a cópia mais recente e conferir o conteúdo.

```
from pathlib import Path

# A cópia mais recente: com a data no formato ISO, a ordem alfabética
# dos nomes é também a ordem cronológica
arquivo = sorted(Path("dados").glob("yahoo_fechamentos_*.csv"))[-1]
precos = pd.read_csv(arquivo, index_col="Date", parse_dates=True)

print(arquivo.name)
print(precos.round(2).head(3))
```

```
print(precos.shape, precos.index.min().date(), precos.index.max().date())
```

```
yahoo_fechamentos_2026-10-10.csv
            ITUB4.SA  PETR4.SA  VALE3.SA    ^BVSP    ^GSPC
Date
2016-01-04      7.23      1.72      6.10  42141.0  2012.66
2016-01-05      7.30      1.67      6.02  42419.0  2016.71
2016-01-06      7.27      1.60      5.58  41773.0  1990.26
(2581, 5) 2016-01-04 2025-12-31
```

Nota

Os preços das ações em 2016 parecem baixos demais para quem acompanhou o mercado: a PETR4 aparece a R$ 1,72. São preços *ajustados*: o Yahoo desconta deles, para trás, todos os dividendos e eventos pagos desde então, de modo que a variação entre duas datas reflita o ganho total do acionista. A Seção 6.3 mostra como conferir isso com os dados oficiais da B3.

4.3 Seleção, dados faltantes e dias sem pregão

Com o índice de datas, `.loc` aceita datas completas, intervalos e até um mês inteiro, como `"2025-12"`.

Listagem 4.4 Selecionando períodos e colunas pela data.

```
ibov = precos["^BVSP"]                          # uma coluna é uma Series
print(ibov.loc["2020-03-09":"2020-03-13"])      # uma semana de março de 2020

print(precos.loc["2025-12", ["PETR4.SA", "VALE3.SA"]].round(2).tail(3))
print(ibov.idxmax().date(), ibov.max())         # data e valor do máximo
```

```
Date
2020-03-09    86067.0
2020-03-10    92214.0
2020-03-11    85171.0
2020-03-12    72583.0
2020-03-13    82678.0
Name: ^BVSP, dtype: float64
            PETR4.SA  VALE3.SA
Date
2025-12-29     28.89     70.15
2025-12-30     28.98     70.00
2025-12-31       NaN       NaN
2025-12-04 164456.0
```

A semana de 9 a 13 de março de 2020 mostra o Ibovespa caindo de 86 067 para 72 583 pontos em três pregões, no auge da crise da covid-19. Repare também no `NaN` (*not a number*) em 31/12/2025: a B3 não tem pregão no último dia do ano, e o pandas marca a ausência de valor com `NaN`.

Dados faltantes merecem diagnóstico antes de qualquer cálculo. A Listagem 4.5 conta os `NaN` de cada coluna e investiga de onde eles vêm.

Listagem 4.5 Diagnóstico e limpeza dos dados faltantes.

```python
print(precos.isna().sum())

# Dias com S&P 500 e sem Ibovespa: feriado no Brasil, pregão nos EUA
so_eua = precos["^BVSP"].isna() & precos["^GSPC"].notna()
print(so_eua.sum(), "dias, por exemplo:", precos.index[so_eua][:2].date)

# Dias com preço das ações e sem Ibovespa: o que aconteceu?
suspeitos = precos["^BVSP"].isna() & precos["PETR4.SA"].notna()
anteriores = precos["PETR4.SA"].shift(1)[suspeitos]
print(suspeitos.sum(), "dias; preço igual ao da véspera:",
      (precos.loc[suspeitos, "PETR4.SA"] == anteriores).sum())

# Preço repetido em dia sem índice: tratamos como dia sem pregão na B3
acoes = ["ITUB4.SA", "PETR4.SA", "VALE3.SA"]
precos.loc[suspeitos, acoes] = None
print(precos[acoes].isna().sum().tolist())
```

```
ITUB4.SA     90
PETR4.SA     90
VALE3.SA     90
^BVSP        98
^GSPC        67
dtype: int64
97 dias, por exemplo: [datetime.date(2016, 1, 25) datetime.date(2016, 2, 8)]
8 dias; preço igual ao da véspera: 8
[98, 98, 98]
```

O diagnóstico revela dois casos diferentes:

- **97 dias com S&P 500 e sem Ibovespa**: feriados no Brasil em que houve pregão nos Estados Unidos (e o contrário explica os `NaN` do S&P 500). É o comportamento esperado quando se juntam mercados com calendários diferentes.
- **8 dias com preço das ações e sem Ibovespa**: em todos eles, o preço das ações é idêntico ao da véspera. Tudo indica que não houve pregão, e que a fonte repetiu o último preço. Mantidos, esses dias criariam retornos iguais a zero que não aconteceram. A listagem os transforma em `NaN`, e as três ações passam a ter exatamente os mesmos 98 dias sem preço que o Ibovespa.

Armadilha comum

Preencher lacunas com o último preço (`ffill`) só para "completar a tabela". Cada preço repetido vira um retorno igual a zero, que nunca aconteceu: a volatilidade fica subestimada e as caudas, mais leves do que são. Para calcular retornos, use em cada série apenas os dias em que ela foi negociada; para comparar séries, use as datas em comum.

4.4 Reamostragem: de diário a mensal

Reamostrar é mudar a frequência de uma série. O método `resample` agrupa as datas por período (`"ME"`, fim do mês; `"YE"`, fim do ano) e aplica uma agregação. Para preços, a agregação natural é o **último** valor de cada período.

Listagem 4.6 Preços mensais e variação anual do Ibovespa.

```
mensal = precos.resample("ME").last()          # último preço de cada mês
print(mensal.round(2).tail(3))

anual = precos["^BVSP"].resample("YE").last()
print((anual.pct_change() * 100).round(1).dropna())
```

```
            ITUB4.SA  PETR4.SA  VALE3.SA     ^BVSP    ^GSPC
Date
2025-10-31     35.42     27.12     60.32  149540.0  6840.20
2025-11-30     37.41     28.98     62.30  159072.0  6849.09
2025-12-31     38.40     28.98     70.00  161125.0  6845.50
Date
2017-12-31    26.9
2018-12-31    15.0
2019-12-31    31.9
2020-12-31     2.9
2021-12-31   -12.1
2022-12-31     5.0
2023-12-31    22.0
2024-12-31   -10.4
2025-12-31    34.0
Freq: YE-DEC, Name: ^BVSP, dtype: float64
```

A segunda parte calcula a variação anual do Ibovespa com o último fechamento de cada ano. Entre 2017 e 2025, o índice teve anos de alta de mais de 30% e anos de queda de mais de 10%: a média de longo prazo esconde uma trajetória bem irregular.

4.5 Janelas móveis: média e volatilidade

Uma *janela móvel* calcula uma estatística sobre os últimos n pregões, dia após dia. A *volatilidade* é o desvio-padrão dos retornos; multiplicada por $\sqrt{252}$, número aproximado de pregões num ano, fica expressa em termos anuais.

Listagem 4.7 Volatilidade móvel de 21 pregões e média móvel de 200.

```
import numpy as np

ibov = precos["^BVSP"].dropna()               # só os dias com pregão na B3
r = np.log(ibov).diff()
vol_21 = r.rolling(21).std() * np.sqrt(252)   # volatilidade anualizada
media_200 = ibov.rolling(200).mean()          # média móvel de 200 pregões

print(vol_21.describe().round(3))
print("Maior volatilidade:", vol_21.idxmax().date(), round(vol_21.max(), 3))
print("Acima da média de 200 pregões:", f"{(ibov > media_200).mean():.1%}")
```

```
count    2462.000
mean        0.203
std         0.118
min         0.067
25%         0.144
50%         0.182
75%         0.229
max         1.293
Name: ^BVSP, dtype: float64
Maior volatilidade: 2020-04-06 1.293
Acima da média de 200 pregões: 67.3%
```

A volatilidade anualizada de 21 pregões ficou, na metade do tempo, abaixo de 18,2%, mas chegou a 129% em abril de 2020. A volatilidade não é constante: tem períodos calmos e períodos agitados, e o Capítulo 5 vai mostrar isso num gráfico.

Contexto de mercado

A alternância entre períodos calmos e agitados é um dos *fatos estilizados* dos retornos, o *agrupamento de volatilidade* (CONT, 2001). Um passeio aleatório com volatilidade constante, como o do Capítulo 3, não o reproduz. Os Módulos 5 e 8 estudam de onde ele vem.

4.6 Junção de fontes: índice e câmbio

O Ibovespa é cotado em reais. Para compará-lo com um índice estrangeiro, é preciso convertê-lo em dólares, com a cotação do câmbio de cada dia. O Banco Central do Brasil publica essa cotação no seu Sistema Gerenciador de Séries Temporais (SGS); a biblioteca `python-bcb` baixa a série pelo número dela.

Listagem 4.8 Baixar a cotação diária do dólar no Banco Central.

```
from datetime import date
from pathlib import Path

from bcb import sgs

# Série 1 do SGS: taxa de câmbio do dólar americano (venda), em R$ por US$
dolar = sgs.get({"dolar": 1}, start="2016-01-01", end="2025-12-31")

arquivo = Path("dados") / f"bcb_dolar_{date.today()}.csv"
dolar.to_csv(arquivo)
print(arquivo, dolar.shape)
```

```
dados/bcb_dolar_2026-10-10.csv (2510, 1)
```

O método `join` junta duas tabelas pelo índice. Com `how="inner"`, ficam só as datas presentes nas duas: os dias com pregão na B3 *e* cotação de câmbio.

Listagem 4.9 Ibovespa em reais e em dólares.

```
arq_dolar = sorted(Path("dados").glob("bcb_dolar_*.csv"))[-1]
dolar = pd.read_csv(arq_dolar, index_col="Date", parse_dates=True)

junto = precos[["^BVSP"]].join(dolar, how="inner").dropna()
junto["ibov_em_dolar"] = junto["^BVSP"] / junto["dolar"]
print(junto.round(2).iloc[[0, -1]])

variacao = junto.iloc[-1] / junto.iloc[0] - 1
print((variacao * 100).round(1))
```

```
               ^BVSP  dolar  ibov_em_dolar
Date
2016-01-04   42141.0   4.04       10434.30
2025-12-30  161125.0   5.50       29282.68
^BVSP           282.3
dolar            36.2
ibov_em_dolar    180.6
```

```
dtype: float64
```

Entre o primeiro pregão de 2016 e o último de 2025, o Ibovespa subiu 282% em reais. No mesmo período, o dólar subiu 36%, e o índice medido em dólares subiu 181%. A moeda em que se mede um resultado faz parte do resultado.

Por fim, a tabela limpa vai para um arquivo novo, que os Capítulos 5 e 6 usam. O arquivo baixado continua como veio.

Listagem 4.10 Gravar os dados limpos num arquivo novo.

```
from datetime import date

# Os dados limpos vão para um arquivo novo; o arquivo baixado fica intocado
limpo = Path("dados") / f"precos_limpos_{date.today()}.csv"
precos.to_csv(limpo)
print(limpo.name, precos.shape)
```

```
precos_limpos_2026-10-10.csv (2581, 5)
```

Resumo do capítulo

- Series (uma coluna com índice) e DataFrame (várias colunas com o mesmo índice).
- `read_csv(..., index_col=..., parse_dates=True)` cria o índice de datas; `.loc["2020-03"]` seleciona um mês.
- Dados faltantes: diagnosticar antes de tratar. Feriados diferentes geram `NaN` legítimos; preços repetidos em dia sem pregão devem ser removidos.
- `resample("ME").last()` passa de diário a mensal; `rolling(n)` calcula estatísticas em janelas móveis.
- `join(..., how="inner")` alinha fontes diferentes pelas datas em comum.
- Dados brutos ficam intocados; o resultado da limpeza vai para um arquivo novo.

Exercícios

Essencial — fixação

Exercício 4.1 Selecione os fechamentos da VALE3 em junho de 2022 e calcule o maior e o menor preço do mês.

Exercício 4.2 Calcule a variação anual das cinco séries a partir do último preço de cada ano. Em que anos o S&P 500 caiu? Dica: o último preço de cada ano sai de `resample("YE").last()`.

Aplicado — o mesmo método num problema novo

Exercício 4.3 Calcule a volatilidade móvel de 63 pregões (cerca de um trimestre) da PETR4 e encontre a data do seu valor máximo. Coincide com a do Ibovespa?

Exercício 4.4 Converta o S&P 500 para reais com a cotação do dólar e compare a variação de 2016 a 2025 do S&P 500 em reais com a do Ibovespa em reais.

Desafio — extensão aberta

Exercício 4.5 O SGS do Banco Central tem milhares de séries. Encontre o número da série da meta da taxa Selic, baixe-a com `python-bcb` e junte-a ao Ibovespa. Como tratar uma série que só muda de valor nas datas das reuniões do Copom?

CAPÍTULO 5

Gráficos e estatística

◎ Objetivos do capítulo

- Construir figuras com o Matplotlib: preços, retornos, histogramas e escalas logarítmicas.
- Calcular estatísticas descritivas dos retornos: média, desvio-padrão, assimetria e curtose.
- Usar as distribuições do SciPy para calcular probabilidades e ajustar modelos.
- Aplicar e interpretar um teste de normalidade.

Este capítulo trabalha com os log-retornos diários do Ibovespa entre 2016 e 2025. Ao final dele, você terá produzido, com os seus próprios dados, a primeira evidência de que os retornos não seguem a distribuição normal: o ponto de partida da Econofísica.

A Listagem 5.1 carrega os dados limpos no Capítulo 4 e calcula os log-retornos do Ibovespa, só nos dias de pregão da B3.

Listagem 5.1 Dados do capítulo: log-retornos diários do Ibovespa.

```python
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy import stats

arquivo = sorted(Path("dados").glob("precos_limpos_*.csv"))[-1]
precos = pd.read_csv(arquivo, index_col="Date", parse_dates=True)
ibov = precos["^BVSP"].dropna()
r = np.log(ibov).diff().dropna()            # log-retornos diários
print(len(r), "retornos, de", r.index.min().date(),
      "a", r.index.max().date())
```

```
2482 retornos, de 2016-01-05 a 2025-12-30
```

5.1 Anatomia de uma figura

No Matplotlib, uma *figura* (`fig`) é a folha inteira, e cada gráfico dentro dela é um *eixo* (`ax`). A função `plt.subplots` cria os dois de uma vez. Os métodos do eixo desenham (`plot`, `hist`) e anotam (`set_title`, `set_ylabel`); `fig.savefig` grava a figura num arquivo.

Listagem 5.2 A primeira figura: o Ibovespa de 2016 a 2025.

```
fig, ax = plt.subplots()
ax.plot(ibov.index, ibov)
ax.set_title("Ibovespa, 2016 a 2025")
ax.set_ylabel("pontos")
fig.savefig("figuras/fig_05_01_ibovespa.pdf")
```



Figura 5.1 O Ibovespa entre 2016 e 2025, em escala linear.

Boa prática

Defina o estilo das figuras num lugar só. A pasta `codigo/` tem um arquivo `matplotlibrc` com o tamanho padrão, as fontes, a grade e as cores de todas as figuras do caderno. O Matplotlib lê esse arquivo automaticamente quando o código roda na pasta. Assim, as listagens tratam só do conteúdo, e todas as figuras saem com a mesma aparência.

5.2 Preços e retornos no tempo

Na escala linear da Figura 5.1, uma queda de 10 mil pontos parece maior em 2025 do que em 2016, embora, em porcentagem, seja muito menor. Na *escala logarítmica*, distâncias verticais iguais correspondem a variações percentuais iguais: cada marca do eixo da Figura 5.2 é o dobro da anterior. Abaixo do preço, o gráfico mostra os log-retornos diários.

Listagem 5.3 Preço em escala logarítmica e log-retornos diários.

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(6.4, 4.4), sharex=True)
```

```
ax1.plot(ibov.index, ibov)
ax1.set_yscale("log")
ax1.set_yticks([40_000, 80_000, 160_000],
               labels=["40 mil", "80 mil", "160 mil"])   # cada marca dobra
ax1.minorticks_off()
ax1.set_title("Ibovespa (escala logarítmica)")
ax2.plot(r.index, r, color="C3", linewidth=0.5)
ax2.set_title("Log-retornos diários")
fig.savefig("figuras/fig_05_02_precos_retornos.pdf")
```

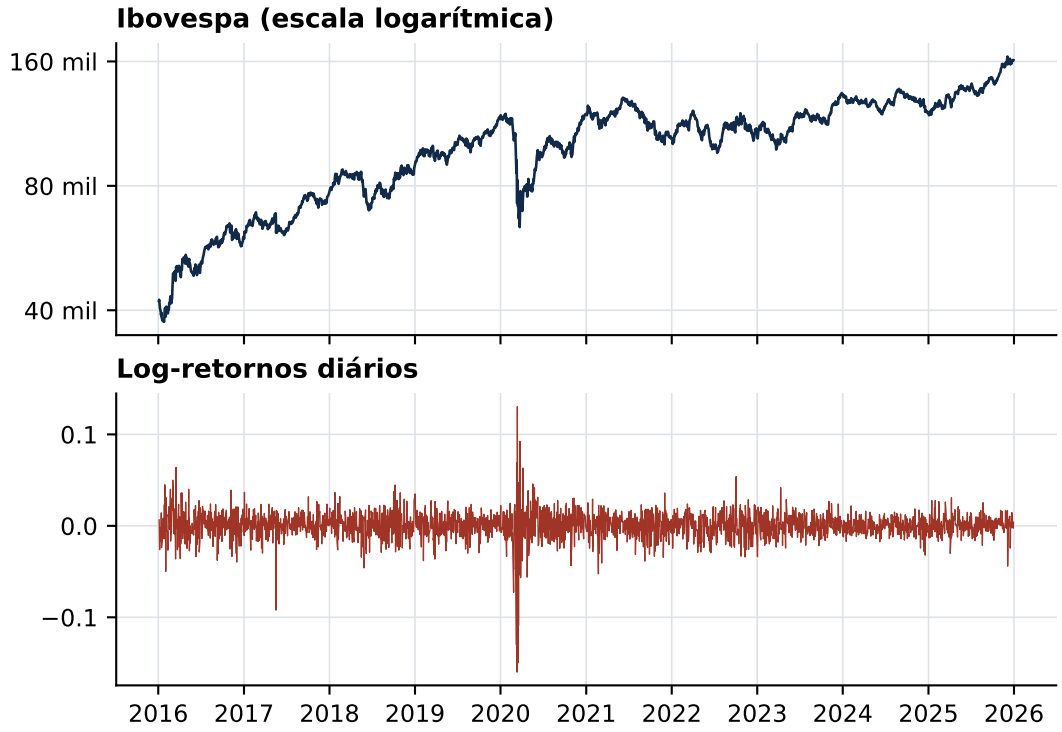


Figura 5.2 Ibovespa em escala logarítmica (acima) e os seus log-retornos diários (abaixo). A volatilidade se concentra em períodos, como março de 2020.

O painel de baixo é a imagem do agrupamento de volatilidade que a Seção 4.5 mediu: longos trechos de retornos pequenos, interrompidos por surtos de retornos grandes, de ambos os sinais. Num passeio aleatório com volatilidade constante, o painel seria uma faixa de largura uniforme.

5.3 Histogramas e escalas logarítmicas

O *histograma* mostra como os retornos se distribuem. Para compará-lo com a distribuição normal, a Listagem 5.4 desenha por cima a densidade de uma normal com a mesma média e o mesmo desvio-padrão dos dados. A função `stats.norm.pdf` do SciPy calcula essa densidade. O mesmo gráfico aparece duas vezes: com o eixo vertical linear e com o eixo vertical logarítmico.

Listagem 5.4 Histograma dos retornos e a normal de mesma média e desvio.

```
x = np.linspace(r.min(), r.max(), 400)
```

```
normal = stats.norm.pdf(x, r.mean(), r.std())    # mesma média e desvio

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(6.4, 3.0))
for ax in (ax1, ax2):
    ax.hist(r, bins=100, density=True, alpha=0.6, label="Ibovespa")
    ax.plot(x, normal, color="C1", label="normal")
ax1.set_title("Escala linear")
ax2.set_title("Escala logarítmica")
ax2.set_yscale("log")
ax2.set_ylim(bottom=0.05)
ax1.legend()
fig.savefig("figuras/fig_05_03_histograma.pdf")
```

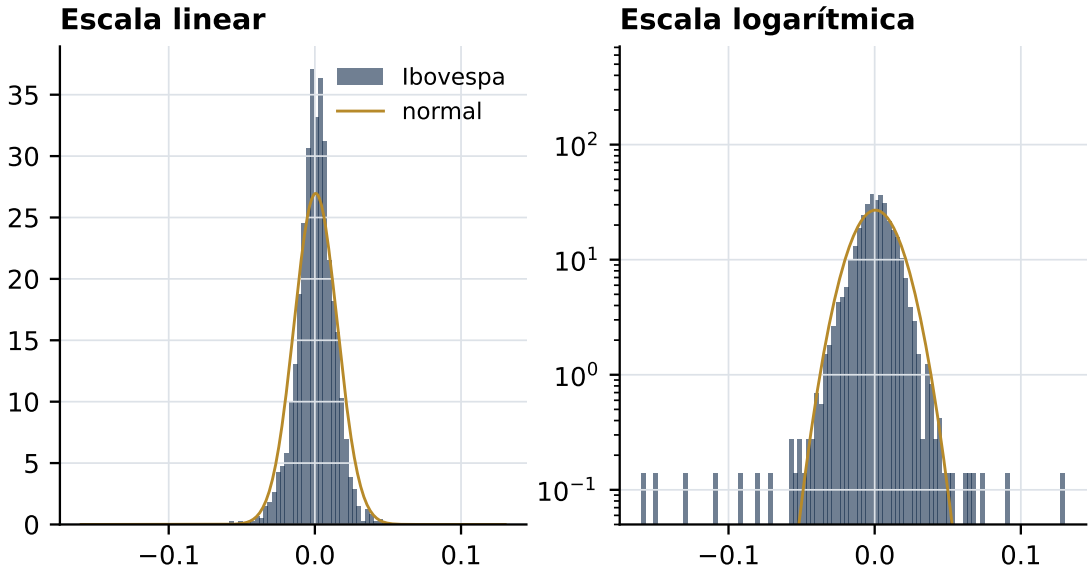


Figura 5.3 Histograma dos log-retornos diários do Ibovespa e a normal de mesma média e desvio-padrão. Na escala logarítmica (à direita), as caudas dos dados ficam muito acima da curva normal.

Na escala linear, a normal parece um ajuste razoável, apenas um pouco mais baixa no centro. A escala logarítmica conta outra história: nas extremidades, onde a normal despenca, os dados continuam aparecendo. Retornos de -10% ou -15%, que para a normal estão a muitos desvios-padrão da média, aconteceram de fato.

Contexto de mercado

Esse excesso de eventos extremos são as *caudas pesadas*, o fato estilizado mais importante dos retornos financeiros (MANDELBROT, 1963; CONT, 2001). O Módulo 2 inteiro é dedicado a medi-las. A escala logarítmica no eixo das frequências é a ferramenta básica para enxergá-las: em escala linear, as caudas simplesmente não aparecem.

5.4 Estatística descritiva

Quatro números resumem uma distribuição de retornos:

- a **média**, o retorno típico;
- o **desvio-padrão**, a dispersão em torno da média, que em finanças se chama volatilidade;
- a **assimetria**, que é negativa quando as quedas extremas são maiores ou mais frequentes que as altas extremas;
- a **curtose em excesso**, que mede o peso das caudas em relação à normal. Na normal ela vale zero; valores positivos indicam caudas mais pesadas.

Listagem 5.5 Estatísticas descritivas dos log-retornos das cinco séries.

```python
# Log-retornos de cada série no seu próprio calendário de pregões
retornos = pd.DataFrame({c: np.log(precos[c].dropna()).diff()
                         for c in precos.columns})

resumo = pd.DataFrame({
    "media_%": retornos.mean() * 100,
    "desvio_%": retornos.std() * 100,
    "assimetria": retornos.apply(lambda s: stats.skew(s.dropna())),
    "curtose_exc": retornos.apply(lambda s: stats.kurtosis(s.dropna())),
})
print(resumo.round(2))
```

```
          media_%  desvio_%  assimetria  curtose_exc
ITUB4.SA     0.07      1.89       -0.50         8.33
PETR4.SA     0.11      2.82       -1.39        18.16
VALE3.SA     0.10      2.54       -0.42        11.34
^BVSP        0.05      1.48       -1.18        17.81
^GSPC        0.05      1.15       -0.68        16.55
```

Todas as séries têm assimetria negativa e curtose em excesso muito acima de zero: de 8 no ITUB4 a 18 na PETR4. Não é uma particularidade do mercado brasileiro: o S&P 500 tem curtose em excesso de 16,55.

Armadilha comum

Calcular os retornos de uma tabela com calendários misturados aplicando direto `np.log(precos).diff()`. Onde há um `NaN`, o retorno do dia seguinte também vira `NaN`, e esse retorno se perde. A Listagem 5.5 calcula os retornos de cada coluna depois de remover os seus próprios `NaN`, e só então junta as colunas.

5.5 Distribuições no SciPy

O módulo `scipy.stats` traz dezenas de distribuições de probabilidade, todas com a mesma interface: `pdf` (densidade), `cdf` (probabilidade acumulada), `sf` (probabilidade da cauda direita, $1-$`cdf`), `ppf` (quantil) e `fit` (ajuste aos dados). A Listagem 5.6 compara, para o Ibovespa, quantos dias tiveram retornos extremos e quantos a normal preveria.

Listagem 5.6 Eventos extremos: dados contra a normal.

```
m, s = r.mean(), r.std()
for k in [3, 4, 5]:
    observados = (np.abs(r - m) > k * s).sum()
    esperados = len(r) * 2 * stats.norm.sf(k)   # sf = 1 - cdf
    print(f"|r - média| > {k} desvios: {observados:3d} dias "
          f"(a normal prevê {esperados:.2f})")

p_normal = stats.norm.cdf(-0.05, m, s)
print(f"Queda de mais de 5% num dia: normal {p_normal:.3%}, "
      f"dados {(r < -0.05).mean():.3%}")
```

```
|r - média| > 3 desvios:  26 dias (a normal prevê 6.70)
|r - média| > 4 desvios:  13 dias (a normal prevê 0.16)
|r - média| > 5 desvios:   8 dias (a normal prevê 0.00)
Queda de mais de 5% num dia: normal 0.032%, dados 0.443%
```

Retornos a mais de 4 desvios-padrão da média aconteceram em 13 dias; a normal prevê 0,16. Quedas de mais de 5% num dia foram 14 vezes mais frequentes do que a normal prevê. A normal não erra por pouco nas caudas: erra por ordens de grandeza.

Uma alternativa simples é a distribuição *t de Student*, cujas caudas são mais pesadas quanto menor o seu parâmetro, os *graus de liberdade*. O método `fit` estima esse parâmetro a partir dos dados.

Listagem 5.7 Ajuste de uma *t* de Student aos retornos.

```
gl, loc, escala = stats.t.fit(r)
print(f"graus de liberdade: {gl:.2f}")
print(f"Queda de mais de 5% pela t ajustada: "
      f"{stats.t.cdf(-0.05, gl, loc, escala):.3%}")
```

```
graus de liberdade: 4.08
Queda de mais de 5% pela t ajustada: 0.344%
```

Com cerca de 4 graus de liberdade, a *t* ajustada atribui às quedas de mais de 5% uma probabilidade de 0,34%, muito mais perto dos 0,44% observados do que os 0,03% da normal.

Outra forma de comparar os dados com a normal é o *gráfico quantil-quantil* (Figura 5.4): cada ponto põe um quantil dos dados contra o quantil correspondente da normal. Se os dados fossem normais, os pontos ficariam sobre a reta.

Listagem 5.8 Gráfico quantil-quantil dos retornos contra a normal.

```
fig, ax = plt.subplots(figsize=(3.6, 3.2))
stats.probplot(r, dist="norm", plot=ax)
ax.get_lines()[0].set(color="C0", markersize=2)
ax.get_lines()[1].set(color="C1")
ax.set_title("Quantis do Ibovespa contra a normal")
ax.set_xlabel("quantis teóricos (normal)")
ax.set_ylabel("quantis observados")
fig.savefig("figuras/fig_05_04_qq.pdf")
```

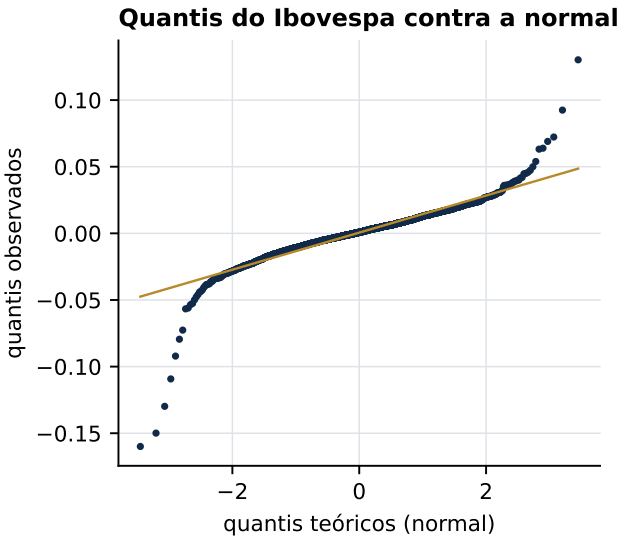


Figura 5.4 Gráfico quantil-quantil dos log-retornos do Ibovespa. No centro, os pontos seguem a reta; nas pontas, se afastam dela: as caudas dos dados são mais pesadas que as da normal.

5.6 Testes de normalidade

Um *teste de hipótese* pergunta se os dados são compatíveis com uma hipótese, aqui a de que vêm de uma distribuição normal. O teste de *Jarque-Bera* combina a assimetria e a curtose da amostra numa estatística; quanto maior ela, mais longe da normal. O *valor-p* é a probabilidade de observar uma estatística tão grande se a hipótese fosse verdadeira.

Listagem 5.9 O teste de Jarque-Bera nos dados e numa amostra normal.

```
jb = stats.jarque_bera(r)
print(f"Ibovespa:       JB = {jb.statistic:9.1f}   p = {jb.pvalue:.3g}")

amostra = np.random.default_rng(42).normal(m, s, size=len(r))
jb_n = stats.jarque_bera(amostra)
print(f"Amostra normal: JB = {jb_n.statistic:9.1f}   p = {jb_n.pvalue:.3g}")
```

```
Ibovespa:       JB =   33372.2   p = 0
Amostra normal: JB =       0.8   p = 0.682
```

Para o Ibovespa, o valor-p é tão pequeno que o computador o arredonda para zero: a normalidade é rejeitada sem nenhuma dúvida. Para uma amostra sorteada de uma normal, com o mesmo tamanho, o valor-p é 0,68, e a normalidade não é rejeitada, como deveria ser. Testar o método com dados de resposta conhecida é a melhor forma de confiar nele.

Armadilha comum

Ler o valor-p como a probabilidade de a hipótese ser verdadeira. Ele é a probabilidade dos dados *supondo* a hipótese. E, com amostras muito grandes, qualquer desvio mínimo leva à rejeição; com amostras pequenas, desvios grandes podem passar despercebidos. Por isso, olhe sempre os gráficos junto com o teste.

Resumo do capítulo

- Matplotlib: `fig, ax = plt.subplots()`; o eixo desenha e anota; `fig.savefig` grava. O estilo fica no `matplotlibrc`.
- Escala logarítmica: no preço, variações percentuais iguais têm alturas iguais; nas frequências, as caudas ficam visíveis.
- Os retornos do Ibovespa têm assimetria negativa ($-1{,}2$) e curtose em excesso próxima de 18; eventos de 4 ou 5 desvios-padrão são dezenas de vezes mais frequentes do que a normal prevê.
- `scipy.stats`: `pdf`, `cdf`, `sf`, `ppf` e `fit`; a t de Student com cerca de 4 graus de liberdade descreve as caudas muito melhor que a normal.
- Jarque-Bera rejeita a normalidade dos retornos; teste o método antes com dados cuja resposta você conhece.

Exercícios

Essencial — fixação

Exercício 5.1 Refaça a Figura 5.3 para a PETR4. A diferença entre os dados e a normal é maior ou menor que no Ibovespa? Compare com a curtose da Listagem 5.5.

Exercício 5.2 Use `stats.norm.ppf` para encontrar o retorno diário que a normal ajustada ao Ibovespa só ultrapassa para baixo em 1% dos dias. Em quantos dias de 2016 a 2025 o Ibovespa caiu mais que isso?

Aplicado — o mesmo método num problema novo

Exercício 5.3 Repita o teste de Jarque-Bera com retornos *mensais* do Ibovespa (some os log-retornos diários de cada mês). A normalidade continua sendo rejeitada? O que isso sugere sobre a forma da distribuição em horizontes mais longos?

Exercício 5.4 Aplique a Listagem 5.6 às 10 000 trajetórias simuladas no Exercício 3.3. Os eventos extremos aparecem na frequência que a normal prevê? Compare com o Ibovespa.

Desafio — extensão aberta

Exercício 5.5 Faça o gráfico da função de sobrevivência empírica das quedas do Ibovespa, $P(-r > x)$, em escala log-log, junto com a da normal e a da t ajustada. Que forma tem a curva dos dados para x grande? Esse gráfico é o ponto de partida do Módulo 2.

CAPÍTULO 6

Dados de mercado e retornos

◎ Objetivos do capítulo

- Conhecer as fontes públicas de dados de mercado e registrar a origem e a data de cada arquivo.
- Ler o arquivo oficial de cotações históricas da B3.
- Conferir uma fonte contra outra e distinguir preço bruto de preço ajustado.
- Calcular retornos simples e logarítmicos e agregá-los no tempo.
- Montar a base de retornos que os próximos cadernos vão usar.

Todo resultado de Econofísica depende dos dados que o produziram. Este capítulo trata os dados como parte do método: de onde vêm, como são lidos, como se confere uma fonte contra outra e como se registra cada passo. Ao final, você terá montado a prática principal do Módulo 0: uma base de retornos de ações e índices brasileiros e internacionais, com origem documentada.

6.1 Fontes públicas e o registro de dados

O caderno usa três fontes, com papéis diferentes (Tabela 6.1).

Tabela 6.1 As três fontes de dados do caderno.

Fonte	O que oferece	Como acessar
B3	Cotações oficiais de todos os papéis negociados na bolsa brasileira, sem ajuste por proventos	Arquivos anuais de séries históricas, de layout fixo, no site da B3 (Seção 6.2)
Banco Central do Brasil	Câmbio, juros, inflação e milhares de outras séries	API de dados abertos do SGS, pela biblioteca `python-bcb` (Listagem 4.8)
Yahoo Finance	Cotações de ações e índices de vários países, com preços ajustados	Pela biblioteca `yfinance` (Listagem 4.2)

A B3 é a fonte oficial do mercado brasileiro, mas os seus arquivos são grandes e trazem apenas preços brutos. O Yahoo Finance é prático, cobre vários países e oferece preços ajustados, mas não é uma fonte oficial: a biblioteca `yfinance` não é afiliada ao Yahoo, e a sua própria documentação lembra que ela se destina a pesquisa e ensino e que a API do Yahoo Finance é para uso pessoal. Por isso, a boa prática é usar o Yahoo pela conveniência e conferi-lo contra a

fonte oficial, como na Seção 6.3.

Armadilha comum

Redistribuir dados sem ler os termos de uso. Cada leitor deste caderno baixa a sua própria cópia dos dados; os arquivos baixados não acompanham o livro. Antes de publicar dados brutos (num repositório público, por exemplo), leia os termos de uso de cada fonte. Resultados derivados, como estatísticas e figuras, costumam ser outra questão, mas a dúvida se resolve nos termos, e não por suposição.

Quem baixa dados de várias fontes precisa saber, meses depois, de onde veio cada arquivo e quando foi baixado. A função `registrar` da Listagem 6.1 mantém um *registro de dados*: um CSV com uma linha por arquivo, com nome, fonte, descrição e data do download (tirada da data de modificação do próprio arquivo). Se o mesmo arquivo for registrado de novo, a linha antiga é substituída.

Listagem 6.1 Um registro de dados: origem e data de cada arquivo.

```python
from datetime import date
from pathlib import Path

import numpy as np
import pandas as pd

REGISTRO = Path("dados") / "registro_dados.csv"


def registrar(arquivo, fonte, descricao):
    """Anota no registro de dados a origem e a data de um arquivo."""
    arquivo = Path(arquivo)
    linha = pd.DataFrame([{
        "arquivo": arquivo.name, "fonte": fonte, "descricao": descricao,
        "baixado_em": date.fromtimestamp(arquivo.stat().st_mtime)}])
    if REGISTRO.exists():                          # substitui a linha antiga
        antigo = pd.read_csv(REGISTRO)
        antigo = antigo[antigo["arquivo"] != arquivo.name]
        linha = pd.concat([antigo, linha])
    linha.to_csv(REGISTRO, index=False)
```

6.2 As séries históricas da B3

A B3 publica, para cada ano, um arquivo com as cotações diárias de todos os papéis negociados. O de 2025 tem 89 MB compactado e quase 800 MB descompactado. A Listagem 6.2 o baixa.

Listagem 6.2 Baixar o arquivo anual de séries históricas da B3.

```python
import urllib.request
from pathlib import Path

url = "https://bvmf.bmfbovespa.com.br/InstDados/SerHist/COTAHIST_A2025.ZIP"
arquivo_b3 = Path("dados") / "COTAHIST_A2025.ZIP"

# O servidor da B3 recusa pedidos sem identificação de navegador (erro 403)
pedido = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
```

```
with urllib.request.urlopen(pedido) as resposta:
    arquivo_b3.write_bytes(resposta.read())
print(arquivo_b3, round(arquivo_b3.stat().st_size / 1e6, 1), "MB")
```

```
dados/COTAHIST_A2025.ZIP 89.1 MB
```

O servidor da B3 recusa pedidos que não se identificam como um navegador, por isso o cabeçalho `User-Agent` na listagem. Sem ele, a resposta é o erro 403 (“proibido”).

O arquivo tem *layout fixo*: cada linha é um registro, e cada campo ocupa sempre as mesmas colunas de caracteres, definidas no documento de layout da B3 (Tabela 6.2). Os preços não têm vírgula: as duas últimas posições são as casas decimais.

Tabela 6.2 Campos do arquivo de séries históricas da B3 usados no caderno (posições contadas a partir de 1, segundo o layout oficial).

Campo	Conteúdo	Posições	Observação
TIPREG	tipo de registro	1–2	01 = cotação
DATA	data do pregão	3–10	AAAAMMDD
CODBDI	código BDI	11–12	02 = lote padrão
CODNEG	código de negociação	13–24	o ticker, como PETR4
TPMERC	tipo de mercado	25–27	010 = mercado à vista
ESPECI	especificação do papel	40–49	EB = ex-bonificação
PREABE a PREMIN	abertura, máxima, mínima	57–95	2 casas decimais
PREULT	preço do último negócio	109–121	2 casas decimais
TOTNEG	número de negócios	148–152	
VOLTOT	volume financeiro	171–188	2 casas decimais

A Listagem 6.3 traduz a tabela em código. A função `ler_cotahist` lê o arquivo compactado **uma linha por vez**, sem descompactá-lo no disco, e só monta os campos das linhas que interessam: as cotações dos tickers pedidos, no lote padrão do mercado à vista.

Listagem 6.3 Uma função que lê o layout fixo da B3.

```
import zipfile

# Posições (início, fim) de cada campo no layout oficial da B3, contadas a
# partir de 1. Os preços têm duas casas decimais implícitas.
CAMPOS = {"data": (3, 10), "codbdi": (11, 12), "ticker": (13, 24),
          "tpmerc": (25, 27), "especi": (40, 49), "abertura": (57, 69),
          "maxima": (70, 82), "minima": (83, 95), "fechamento": (109, 121),
          "negocios": (148, 152), "volume": (171, 188)}
PRECOS = ["abertura", "maxima", "minima", "fechamento", "volume"]


def ler_cotahist(caminho, tickers):
    """Lê do arquivo anual da B3 as cotações à vista (lote padrão)."""
    registros = []
    with zipfile.ZipFile(caminho) as z, z.open(z.namelist()[0]) as f:
        for bruta in f:                          # uma linha por vez
            linha = bruta.decode("latin-1")
            if linha[:2] != "01" or linha[12:24].strip() not in tickers:
                continue                         # 00 e 99: cabeçalho, rodapé
            campos = {nome: linha[i - 1:j].strip()
                      for nome, (i, j) in CAMPOS.items()}
            if campos["codbdi"] == "02" and campos["tpmerc"] == "010":
                registros.append(campos)
```

```
    tabela = pd.DataFrame(registros)
    tabela["data"] = pd.to_datetime(tabela["data"], format="%Y%m%d")
    tabela[PRECOS] = tabela[PRECOS].astype("int64") / 100
    tabela["negocios"] = tabela["negocios"].astype("int64")
    tabela = tabela.sort_values(["data", "ticker"], ignore_index=True)
    return tabela.drop(columns=["codbdi", "tpmerc"])
```

Boa prática

Diante de um arquivo grande, filtre antes de montar a tabela. Ler os quase 800 MB para um DataFrame e só depois filtrar três tickers exigiria vários gigabytes de memória. Percorrer as linhas e guardar só as que interessam leva poucos segundos e quase nenhuma memória.

Listagem 6.4 As cotações de 2025 de PETR4, VALE3 e ITUB4.

```
arquivo_b3 = Path("dados") / "COTAHIST_A2025.ZIP"
b3 = ler_cotahist(arquivo_b3, {"PETR4", "VALE3", "ITUB4"})

print(b3.shape)
print(b3[["data", "ticker", "fechamento", "negocios"]].head(3))
# Primeiro pregão de cada mês com o ITUB4 marcado ex-bonificação (EB)
eb = b3[b3["especi"].str.contains("EB")]
print(eb.groupby(eb["data"].dt.month)["data"].min())
```

```
(750, 9)
        data ticker  fechamento  negocios
0 2025-01-02  ITUB4       30.57     57666
1 2025-01-02  PETR4       36.77     72178
2 2025-01-02  VALE3       54.25     52924
data
3    2025-03-18
12   2025-12-26
Name: data, dtype: datetime64[us]
```

São 250 pregões para cada um dos três papéis. A última parte da saída mostra algo que vai ser importante na próxima seção: em 2025, o ITUB4 aparece com a especificação `EB` (*ex-bonificação*) a partir de 18 de março e de 26 de dezembro.

6.3 Conferindo fontes: preço bruto e preço ajustado

A B3 informa o preço efetivamente negociado em cada pregão: o preço *bruto*. Quando uma empresa paga dividendos ou distribui ações (bonificação, desdobramento), o preço cai na data em que o direito deixa de acompanhar a ação, sem que o acionista tenha perdido dinheiro. Um preço *ajustado* corrige essas quedas, recalculando para trás todos os preços anteriores ao evento.

O Yahoo oferece as duas versões: `Close`, ajustado apenas por desdobramentos e bonificações, e `Adj Close`, ajustado também por dividendos. A Listagem 6.5 baixa as duas para 2025, e a Listagem 6.6 confronta o `Close` com o preço da B3.

Listagem 6.5 Os mesmos papéis em 2025, sem o ajuste por dividendos.

```python
from datetime import date
from pathlib import Path

import yfinance as yf

# Os mesmos papéis em 2025, agora SEM o ajuste por proventos
papeis = ["PETR4.SA", "VALE3.SA", "ITUB4.SA"]
brutos = yf.download(papeis, start="2025-01-01", end="2026-01-01",
                     auto_adjust=False, progress=False)

arquivo = Path("dados") / f"yahoo_2025_sem_ajuste_{date.today()}.csv"
brutos[["Close", "Adj Close"]].to_csv(arquivo)
print(arquivo, brutos["Close"].shape)
```

```
dados/yahoo_2025_sem_ajuste_2026-10-10.csv (250, 3)
```

Listagem 6.6 Conferindo o Yahoo contra a B3.

```python
arquivo = sorted(Path("dados").glob("yahoo_2025_sem_ajuste_*.csv"))[-1]
yahoo = pd.read_csv(arquivo, header=[0, 1], index_col=0, parse_dates=True)
def sem_sufixo(coluna):
    return coluna.removesuffix(".SA")           # "PETR4.SA" -> "PETR4"


fech_yahoo = yahoo["Close"].rename(columns=sem_sufixo)
ajust_yahoo = yahoo["Adj Close"].rename(columns=sem_sufixo)

fech_b3 = b3.pivot(index="data", columns="ticker", values="fechamento")
diferenca = (fech_b3 - fech_yahoo).abs()
print("Pregões em que B3 e Yahoo diferem em mais de R$ 0,005:")
print((diferenca > 0.005).sum())
print("Razão Yahoo/B3 do ITUB4 ao longo de 2025:")
print((fech_yahoo["ITUB4"] / fech_b3["ITUB4"]).round(4).drop_duplicates())
print("Ajustado / sem ajuste no primeiro pregão de 2025:")
print((ajust_yahoo / fech_yahoo).iloc[0].round(4))
```

```
Pregões em que B3 e Yahoo diferem em mais de R$ 0,005:
ticker
ITUB4    247
PETR4      2
VALE3      2
dtype: int64
Razão Yahoo/B3 do ITUB4 ao longo de 2025:
Date
2025-01-02    0.8826
2025-03-18    0.9709
2025-12-26    1.0000
Name: ITUB4, dtype: float64
Ajustado / sem ajuste no primeiro pregão de 2025:
Ticker
ITUB4    0.8625
PETR4    0.8443
VALE3    0.8594
Name: 2025-01-02 00:00:00, dtype: float64
```

A conferência separa três situações:

- **PETR4 e VALE3**: as duas fontes coincidem em 248 dos 250 pregões. Nos dois restantes de cada papel, diferem em alguns centavos. A causa não é evidente nos arquivos, e

diferenças assim são justamente o que a conferência existe para encontrar e documentar.

- **ITUB4**: a razão entre o Yahoo e a B3 é 0,8826 até 17 de março, 0,9709 até 23 de dezembro e 1 depois disso. São as bonificações marcadas com `EB` pela B3: o Yahoo divide os preços anteriores por 1,1 e por 1,03 (e $1/(1{,}1 \times 1{,}03) = 0{,}8826$).
- **Dividendos**: no primeiro pregão de 2025, o `Adj Close` é de 14% a 16% menor que o `Close`. É o efeito acumulado dos dividendos pagos desde então.

Armadilha comum

Esquecer que o preço ajustado muda com o tempo. O ajuste é recalculado a cada novo dividendo pago, para trás, desde o início da série. Se você baixar os mesmos preços ajustados daqui a um ano, os valores de 2025 estarão diferentes dos de hoje. Por isso as cópias têm a data do download no nome, e por isso o registro de dados guarda essa data. Os *retornos* calculados com uma série ajustada consistente não mudam; os *preços*, sim.

Com as conferências feitas, a Listagem 6.7 registra os quatro arquivos baixados neste caderno.

Listagem 6.7 O registro dos arquivos baixados.

```
for padrao, fonte, descricao in [
    ("yahoo_fechamentos_*.csv", "Yahoo Finance", "fechamentos ajustados"),
    ("bcb_dolar_*.csv", "Banco Central, SGS 1", "dólar, venda"),
    ("COTAHIST_A2025.ZIP", "B3", "séries históricas 2025"),
    ("yahoo_2025_sem_ajuste_*.csv", "Yahoo Finance", "2025, sem ajuste"),
]:
    registrar(sorted(Path("dados").glob(padrao))[-1], fonte, descricao)

print(pd.read_csv(REGISTRO)[["arquivo", "fonte", "baixado_em"]]
      .to_string(index=False))
```

```
                            arquivo                fonte baixado_em
   yahoo_fechamentos_2026-10-10.csv        Yahoo Finance 2026-10-10
           bcb_dolar_2026-10-10.csv Banco Central, SGS 1 2026-10-10
                 COTAHIST_A2025.ZIP                   B3 2026-10-10
yahoo_2025_sem_ajuste_2026-10-10.csv        Yahoo Finance 2026-10-10
```

6.4 Retornos simples e logarítmicos

Os dois retornos já apareceram no Capítulo 2. Para um preço P_t:

$$r_t = \frac{P_t}{P_{t-1}} - 1 \qquad \text{e} \qquad \ell_t = \ln \frac{P_t}{P_{t-1}} = \ln(1 + r_t).$$

No pandas, `pct_change` calcula o retorno simples, e `np.log(...).diff()`, o logarítmico.

Listagem 6.8 Retornos simples e logarítmicos da PETR4 em março de 2020.

```
arquivo = sorted(Path("dados").glob("precos_limpos_*.csv"))[-1]
precos = pd.read_csv(arquivo, index_col="Date", parse_dates=True)
petr4 = precos["PETR4.SA"].dropna()

```

```
simples = petr4.pct_change()                  # P(t) / P(t-1) - 1
logaritmico = np.log(petr4).diff()            # ln P(t) - ln P(t-1)
comparacao = pd.DataFrame({"simples": simples, "log": logaritmico}).dropna()
print(comparacao.loc["2020-03-06":"2020-03-12"].round(4))
print((comparacao["simples"] - comparacao["log"]).abs().max().round(4))
```

```
            simples     log
Date
2020-03-06  -0.0973 -0.1023
2020-03-09  -0.2970 -0.3524
2020-03-10   0.0941  0.0899
2020-03-11  -0.0974 -0.1025
2020-03-12  -0.2050 -0.2295
0.0554
```

Em dias normais, os dois quase coincidem. Em 9 de março de 2020, a PETR4 caiu 29,7%, e o log-retorno foi de $-35{,}2\%$: a diferença chega a 5,5 pontos percentuais justamente nos dias extremos, que são os que a Econofísica mais estuda. Cada retorno tem uma propriedade que o outro não tem:

- o **retorno simples** se agrega entre ativos: o retorno de uma carteira é a média dos retornos simples dos ativos, ponderada pelo peso de cada um;
- o **log-retorno** se agrega no tempo: o log-retorno de um período é a soma dos log-retornos diários.

Como a Econofísica estuda sobretudo a dinâmica de uma série no tempo, o log-retorno é o padrão nos próximos cadernos.

6.5 Agregação temporal

A propriedade de soma tem uma consequência prática: para passar de retornos diários a mensais ou anuais, basta somar.

Listagem 6.9 Soma de log-retornos, retornos mensais e volatilidade anualizada.

```
# Log-retornos se somam no tempo: a soma dos diários dá o do período todo
total_somado = logaritmico.sum()
total_direto = np.log(petr4.iloc[-1] / petr4.iloc[0])
print(round(total_somado, 6), round(total_direto, 6))

# Do diário ao mensal: soma dentro de cada mês
mensal = logaritmico.resample("ME").sum()
print(mensal.loc["2025-10":].round(4))

# Volatilidade anualizada pela regra da raiz do tempo (supõe independência)
print(f"{logaritmico.std() * np.sqrt(252):.1%} ao ano a partir dos diários")
print(f"{mensal.std() * np.sqrt(12):.1%} ao ano a partir dos mensais")
```

```
2.823657 2.823657
Date
2025-10-31   -0.0559
2025-11-30    0.0663
2025-12-31    0.0001
Freq: ME, Name: PETR4.SA, dtype: float64
44.7% ao ano a partir dos diários
43.3% ao ano a partir dos mensais
```

A soma dos log-retornos diários da PETR4 entre 2016 e 2025 coincide, até a sexta casa decimal, com o logaritmo da razão entre o último e o primeiro preço. Os retornos mensais saem de `resample("ME").sum()`.

A última parte compara duas formas de anualizar a volatilidade: a partir dos retornos diários, multiplicando por $\sqrt{252}$, e a partir dos mensais, multiplicando por $\sqrt{12}$. As duas dão valores próximos, 44,7% e 43,3% ao ano. A regra da raiz do tempo vale quando os retornos de dias diferentes são independentes.

Contexto de mercado

A regra da raiz do tempo é uma lei de escala: a volatilidade cresce como $T^{1/2}$ com o horizonte T. O expoente $1/2$ é o do passeio aleatório. Quando os retornos têm memória, o expoente muda, e medir esse expoente é o tema do Módulo 5, com o expoente de Hurst.

6.6 Prática principal: a base de dados do curso

A prática principal do Módulo 0 reúne tudo o que o caderno ensinou numa base de log-retornos diários: as cinco séries do Yahoo, já limpas, mais o Ibovespa convertido em dólares com a cotação do Banco Central. Cada série fica no seu próprio calendário de pregões, e o arquivo resultante entra no registro de dados.

Listagem 6.10 A base de log-retornos diários do Módulo 0.

```
# Prática principal: base de log-retornos diários, cada série no seu
# próprio calendário, mais o Ibovespa convertido em dólares
arq_dolar = sorted(Path("dados").glob("bcb_dolar_*.csv"))[-1]
dolar = pd.read_csv(arq_dolar, index_col="Date", parse_dates=True)["dolar"]

ibov = precos["^BVSP"].dropna()
ibov_usd = (ibov / dolar).dropna()      # datas com as duas cotações

series = {c: precos[c].dropna() for c in precos.columns}
series["IBOV_USD"] = ibov_usd
base = pd.DataFrame({nome: np.log(s).diff() for nome, s in series.items()})

saida = Path("dados") / f"base_retornos_m0_{date.today()}.csv"
base.to_csv(saida)
registrar(saida, "calculado", "log-retornos diários, 2016-2025")
print(saida.name, base.shape)
print(base.count())
```

```
base_retornos_m0_2026-10-10.csv (2580, 6)
ITUB4.SA    2482
PETR4.SA    2482
VALE3.SA    2482
^BVSP       2482
^GSPC       2513
IBOV_USD    2482
dtype: int64
```

As ações e o Ibovespa têm 2 482 retornos cada, nas mesmas datas, graças à limpeza da Listagem 4.5; o S&P 500, no calendário dos Estados Unidos, tem 2 513. A Listagem 6.11 resume a base numa tabela, e a Figura 6.1 mostra o crescimento de 100 investidos em cada série no

início de 2016.

Listagem 6.11 Resumo da base: retorno e volatilidade anualizados, extremos e curtose.

```
tabela = pd.DataFrame({
    "dias": base.count(),
    "media_aa_%": base.mean() * 252 * 100,      # aa: ao ano
    "vol_aa_%": base.std() * np.sqrt(252) * 100,
    "pior_%": base.min() * 100,
    "melhor_%": base.max() * 100,
    "curtose": base.kurt(),                     # curtose em excesso
})
print(tabela.round(1))
```

```
          dias  media_aa_%  vol_aa_%  pior_%  melhor_%  curtose
ITUB4.SA  2482        16.9      30.0   -19.8      10.5      8.3
PETR4.SA  2482        28.7      44.7   -35.2      20.1     18.2
VALE3.SA  2482        24.8      40.3   -28.2      19.4     11.4
^BVSP     2482        13.6      23.5   -16.0      13.0     17.8
^GSPC     2513        12.3      18.2   -12.8       9.1     16.6
IBOV_USD  2482        10.5      31.8   -20.4      16.1     14.8
```

Listagem 6.12 Base 100 de todas as séries.

```
import matplotlib.pyplot as plt

# Crescimento de 100 investidos no início: exp(soma acumulada dos
# log-retornos)
acumulado = 100 * np.exp(base.fillna(0).cumsum())

fig, ax = plt.subplots(figsize=(6.4, 3.6))
colunas = ["^BVSP", "IBOV_USD", "^GSPC", "PETR4.SA", "VALE3.SA", "ITUB4.SA"]
for coluna in colunas:
    ax.plot(acumulado.index, acumulado[coluna], label=coluna)
ax.set_yscale("log")
ax.set_yticks([100, 200, 400, 800, 1600], labels=["100", "200", "400",
                                                  "800", "1600"])
ax.minorticks_off()
ax.set_title("Base 100 em janeiro de 2016 (escala logarítmica)")
ax.legend(ncol=3)
fig.savefig("figuras/fig_06_01_base100.pdf")
```

Nota

A tabela mostra a média anualizada dos log-retornos, e não o retorno anual composto. A média anual em porcentagem e a figura contam a mesma história de formas diferentes: na figura, a PETR4 multiplicou o valor investido por cerca de 17 vezes em dez anos. O preço negociado subiu bem menos, de R$ 6,87 no primeiro pregão de 2016 para R$ 30,82 no último de 2025 (cerca de 4,5 vezes); o restante vem dos dividendos, que o preço ajustado incorpora.

A base deixa visíveis, em números, os temas dos próximos cadernos. A curtose em excesso fica entre 8 e 18 para todas as séries: caudas pesadas, tema do Módulo 2. A volatilidade do Ibovespa medido em dólares (31,8% ao ano) é bem maior que a medida em reais (23,5%): a moeda é uma fonte de risco à parte. E os piores dias de quatro das seis séries caem em março

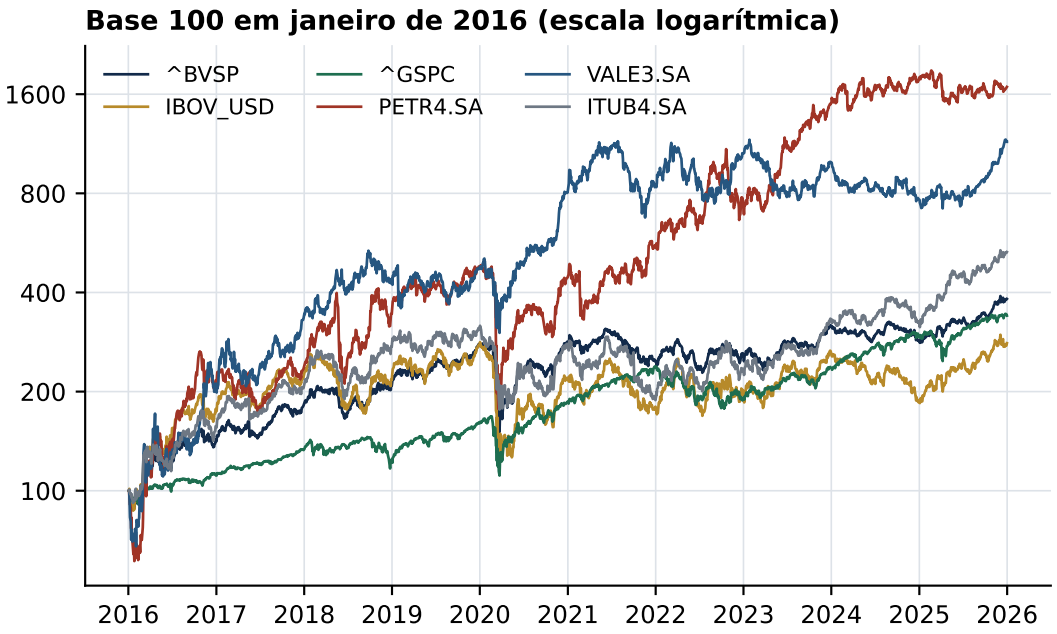


Figura 6.1 Crescimento de 100 investidos em janeiro de 2016 em cada série, em escala logarítmica. As ações usam preços ajustados, que incluem os dividendos.

de 2020; o da VALE3 é 28 de janeiro de 2019, logo depois do rompimento da barragem de Brumadinho, e o do ITUB4 é o tema do Exercício 6.5. Eventos extremos como esses são o assunto do Módulo 9.

6.7 Lista de verificação de reprodutibilidade

Antes de dar por terminada qualquer análise do curso, confira:

1. O ambiente tem as versões do `requirements.txt` (Listagem 1.5).
2. Cada arquivo de dados tem a data do download no nome e uma linha no registro de dados.
3. Os dados brutos não foram editados; a limpeza está no código, e o resultado, num arquivo novo.
4. Os dados faltantes foram diagnosticados antes de tratados.
5. Quando há mais de uma fonte, uma foi conferida contra a outra, e as diferenças estão documentadas.
6. Toda simulação usa um gerador com semente declarada.
7. O notebook (ou o script) roda do início ao fim, num kernel novo, e reproduz todos os números do relatório.

Resumo do capítulo

- Três fontes: B3 (oficial, preços brutos), Banco Central (câmbio e indicadores) e Yahoo Finance (prático, preços ajustados, não oficial). Leia os termos de uso antes de redistribuir dados.
- O arquivo da B3 tem layout fixo: cada campo em posições fixas, preços com duas casas decimais implícitas. Filtre linha a linha.
- Preço bruto, preço ajustado por eventos e preço ajustado por dividendos são coisas diferentes; o ajustado muda a cada novo provento.
- Retorno simples se agrega entre ativos; log-retorno se agrega no tempo.
- A regra da raiz do tempo supõe retornos independentes.
- A base de log-retornos do Módulo 0 está em `dados/`, registrada, e é o ponto de partida dos próximos cadernos.

Exercícios

Essencial — fixação

Exercício 6.1 Use `ler_cotahist` para ler as cotações de 2025 de BBDC4 e ABEV3. Quantos pregões tem cada uma?

Exercício 6.2 Calcule o retorno de 2025 da VALE3 com os preços brutos da B3 e com o `Adj Close` do Yahoo. Explique a diferença.

Aplicado — o mesmo método num problema novo

Exercício 6.3 Monte uma carteira com pesos iguais de PETR4, VALE3 e ITUB4, rebalanceada diariamente, e calcule o seu retorno diário com retornos *simples*. Compare a volatilidade anualizada da carteira com a de cada ação.

Exercício 6.4 Baixe o arquivo da B3 de 2024 e confira os preços do Yahoo (`Close`) contra a B3 para os mesmos três papéis, como na Listagem 6.6. Houve eventos em 2024?

Desafio — extensão aberta

Exercício 6.5 O ITUB4 teve um log-retorno de $-19{,}8\%$ em 4 de outubro de 2021, o seu pior dia na base. Foi uma queda de mercado ou um evento corporativo que o ajuste do Yahoo não corrigiu? Investigue com o arquivo da B3 de 2021 (campo ESPECI) e com o Ibovespa no mesmo dia, e decida se esse dia deve ficar na base.

Referências do Caderno 0

As referências a seguir reúnem as fontes citadas neste caderno e leituras recomendadas para aprofundamento, organizadas por tema. As datas de acesso aos recursos eletrônicos referem-se a 10 de outubro de 2026.

Linguagem Python e documentação

PYTHON SOFTWARE FOUNDATION. **The Python language reference**: versão 3.12. Disponível em: https://docs.python.org/3.12/reference/. Acesso em: 10 out. 2026.

VAN ROSSUM, G.; WARSAW, B.; COGHLAN, N. **PEP 8 – style guide for Python code**. Python Enhancement Proposals, 2001. Disponível em: https://peps.python.org/pep-0008/. Acesso em: 10 out. 2026.

NUMPY DEVELOPERS. **NumPy documentation**. Disponível em: https://numpy.org/doc/. Acesso em: 10 out. 2026.

PANDAS DEVELOPMENT TEAM. **pandas documentation**. Disponível em: https://pandas.pydata.org/docs/. Acesso em: 10 out. 2026.

SCIPY DEVELOPERS. **SciPy documentation**. Disponível em: https://docs.scipy.org/doc/scipy/. Acesso em: 10 out. 2026.

MATPLOTLIB DEVELOPMENT TEAM. **Matplotlib documentation**. Disponível em: https://matplotlib.org/stable/. Acesso em: 10 out. 2026.

PROJECT JUPYTER. **JupyterLab documentation**. Disponível em: https://jupyterlab.readthedocs.io/. Acesso em: 10 out. 2026.

Bibliotecas científicas: artigos de referência

HARRIS, C. R. *et al.* Array programming with NumPy. **Nature**, v. 585, n. 7825, p. 357–362, 2020.

HUNTER, J. D. Matplotlib: a 2D graphics environment. **Computing in Science & Engineering**, v. 9, n. 3, p. 90–95, 2007.

McKINNEY, W. Data structures for statistical computing in Python. In: PROCEEDINGS OF THE 9TH PYTHON IN SCIENCE CONFERENCE (SciPy 2010). Austin, 2010. p. 51–56.

VIRTANEN, P. *et al.* SciPy 1.0: fundamental algorithms for scientific computing in Python. **Nature Methods**, v. 17, n. 3, p. 261–272, 2020.

Livros de programação e de Python para finanças

HILPISCH, Y. **Python for finance**: mastering data-driven finance. 2. ed. Sebastopol: O'Reilly, 2018.

Python aplicado a finanças quantitativas; complementa os Capítulos 4 a 6.

McKINNEY, W. **Python for data analysis**: data wrangling with pandas, NumPy, and Jupyter. 3. ed. Sebastopol: O'Reilly, 2022.

Escrito pelo criador do pandas; a referência mais completa para os Capítulos 3 e 4. Há tradução em português pela Novatec.

MENEZES, N. N. C. **Introdução à programação com Python**: algoritmos e lógica de programação para iniciantes. 3. ed. São Paulo: Novatec, 2019.

Referência em português para quem nunca programou; apoio ao Capítulo 2.

VANDERPLAS, J. **Python data science handbook**: essential tools for working with data. Sebastopol: O'Reilly, 2016.

Panorama de NumPy, pandas e Matplotlib em um único volume.

Econofísica e fatos estilizados

BACHELIER, L. Théorie de la spéculation. **Annales scientifiques de l'École Normale Supérieure**, série 3, v. 17, p. 21–86, 1900.

A origem do passeio aleatório em finanças (Capítulo 3).

CONT, R. Empirical properties of asset returns: stylized facts and statistical issues. **Quantitative Finance**, v. 1, n. 2, p. 223–236, 2001.

O catálogo dos fatos estilizados que aparecem nos Capítulos 4 e 5.

MANDELBROT, B. The variation of certain speculative prices. **The Journal of Business**, v. 36, n. 4, p. 394–419, 1963.

O primeiro estudo sistemático das caudas pesadas dos preços.

MANTEGNA, R. N.; STANLEY, H. E. **An introduction to econophysics**: correlations and complexity in finance. Cambridge: Cambridge University Press, 1999.

Texto de referência da área; leitura natural para os próximos cadernos.

Fontes de dados e bibliotecas de acesso

B3. **Séries históricas**: cotações históricas do mercado à vista e layout do arquivo. Disponível em: https://www.b3.com.br/pt_br/market-data-e-indices/servicos-de-dados/market-data/historico/mercado-a-vista/series-historicas/. Acesso em: 10 out. 2026.

BANCO CENTRAL DO BRASIL. **Sistema Gerenciador de Séries Temporais (SGS)**: série 1, taxa de câmbio do dólar americano (venda). Disponível em: https://api.bcb.gov.br/dados/serie/bcdata.sgs.1/dados?formato=json. Acesso em: 10 out. 2026.

AROUSSI, R. **yfinance**: download market data from Yahoo! Finance API. Disponível em: https://github.com/ranaroussi/yfinance. Acesso em: 10 out. 2026.

Biblioteca não oficial, não afiliada ao Yahoo; leia os termos de uso indicados na documentação.

FREITAS, W. **python-bcb**. Biblioteca em Python para acesso aos dados do Banco Central do Brasil. Disponível em: https://github.com/wilsonfreitas/python-bcb. Acesso em: 10 out. 2026.

CADERNO

1

Fundamentos da Econofísica

Módulo 1 do curso · 6 horas

Apresentação do Caderno 1

O Caderno 0 terminou com uma constatação feita nos seus próprios dados: os retornos do Ibovespa não seguem a distribuição normal. Este caderno pergunta o que fazer com isso.

A resposta passa pela história de uma ideia. Em 1900, Louis Bachelier propôs que os preços se movem ao acaso, como um passeio aleatório. Sessenta anos depois, essa ideia virou a base da teoria financeira, apoiada na hipótese de que os mercados são eficientes. Quase ao mesmo tempo, Benoit Mandelbrot mostrou que os preços reais têm variações extremas demais para o modelo gaussiano. A Econofísica nasceu, nos anos 1990, da decisão de levar essas anomalias a sério, com as ferramentas da física estatística.

Os cinco capítulos seguem esse caminho. O primeiro apresenta a área e prepara os dados. O segundo conta a história, de Bachelier a Mandelbrot. O terceiro discute a hipótese dos mercados eficientes e a testa nos dados. O quarto reúne os *fatos estilizados*, as regularidades estatísticas que todo modelo de mercado precisa reproduzir e que os próximos cadernos estudam um a um. O quinto apresenta as críticas que a Econofísica recebe e o diálogo com a econometria financeira.

É um caderno mais conceitual que o anterior, mas não menos prático: cada afirmação sobre os mercados vem acompanhada do código que a confere, com mais de 75 anos de dados do S&P 500 e mais de 25 anos do Ibovespa.

O autor
2026

Ficha do Módulo 1

Esta ficha reproduz o Módulo 1 do projeto pedagógico do curso e mostra onde cada item é tratado neste caderno.

Módulo	1 — Fundamentos da Econofísica
Eixo	Fundamentos: o que é a Econofísica e o que a distingue da teoria financeira clássica
Carga horária	6 horas: 4 teóricas e 2 práticas
Pré-requisitos	Módulo 0 (Python, NumPy, pandas, Matplotlib e SciPy)
Prática principal	Comparar os retornos de um índice com uma amostra gaussiana de mesma média e variância: série temporal, histograma, gráfico quantil-quantil e autocorrelações (Seção 4.3).

Objetivos do módulo. Ao final, o aluno deve ser capaz de:

- explicar a origem, os objetos de estudo e os métodos da Econofísica (Capítulos 1 e 2);
- relacionar a hipótese dos mercados eficientes ao modelo de passeio aleatório (Capítulo 3);
- identificar os fatos estilizados num conjunto de dados real (Capítulo 4).

Tabela 1 Itens da ementa do Módulo 1 e capítulos deste caderno.

Item da ementa	Capítulo	Horas
O que é a Econofísica e como ela se relaciona com a economia e as finanças	1	1,0
Breve histórico: Bachelier, Mandelbrot e a chegada da física estatística aos mercados	2	1,0
Hipótese dos mercados eficientes e passeio aleatório	3	1,5
O modelo gaussiano de retornos e por que ele é a referência; panorama dos fatos estilizados	4	2,0
Críticas à Econofísica e diálogo com a econometria financeira	5	0,5
Total		**6,0**

CAPÍTULO 1

O que é a Econofísica

◎ Objetivos do capítulo

- Definir a Econofísica e situá-la entre a física, a economia e as finanças.
- Reconhecer os seus objetos de estudo e os seus métodos.
- Entender o que a abordagem da física acrescenta, e o que não pretende substituir.
- Baixar e preparar os dados usados em todo o caderno.

1.1 Uma definição

A *Econofísica* é a área que aplica métodos da física, sobretudo da física estatística e da teoria dos sistemas complexos, a problemas da economia e das finanças. O termo se consolidou na segunda metade dos anos 1990 e é costumeiramente atribuído a H. Eugene Stanley. Um dos primeiros livros a organizar a área foi o de Mantegna e Stanley (MANTEGNA; STANLEY, 1999).

A definição diz menos sobre os temas, que a economia e as finanças estudam há muito tempo, e mais sobre a forma de estudá-los. A física estatística nasceu para explicar como o comportamento coletivo de um gás, de um ímã ou de um fluido surge da interação de um número enorme de partículas, sem acompanhar cada uma delas. Um mercado financeiro também é um sistema com um número enorme de participantes, cujas decisões interagem e produzem um resultado coletivo: o preço. A pergunta da Econofísica é quanto da estatística dos preços pode ser entendida com as ideias que funcionaram para esses sistemas físicos.

1.2 Por que físicos olharam para os mercados

Três características dos mercados financeiros os tornaram atraentes para a física.

Dados abundantes. Poucos sistemas complexos são tão bem medidos quanto um mercado. Cada negócio tem preço, quantidade e horário registrados, e séries diárias de índices cobrem décadas. Este caderno usa mais de 19 mil pregões do S&P 500. Com tantos dados, é possível medir propriedades estatísticas finas, como a forma das caudas de uma distribuição, e confrontá-las com modelos.

Fenômenos coletivos. Os mercados exibem comportamentos que não se explicam olhando um participante isolado: pânicos, bolhas, períodos de calma e de agitação. Na física, fenômenos assim são estudados com modelos de muitos agentes em interação, e é exatamente esse tipo de modelo que o Módulo 8 constrói.

Leis de escala. A física dos fenômenos críticos mostrou que sistemas muito diferentes podem obedecer às mesmas leis de potência, com os mesmos expoentes. Encontrar leis de escala nos dados financeiros, e perguntar se elas são *universais*, isto é, as mesmas para mercados e ativos diferentes, é um dos programas centrais da área.

1.3 Objetos de estudo e métodos

A Tabela 1.1 resume os métodos da Econofísica que este curso cobre e a pergunta que cada um ajuda a responder.

Tabela 1.1 Métodos da Econofísica, as perguntas que respondem e o módulo do curso em que aparecem.

Método	Pergunta	Módulo
Distribuições de cauda pesada	Com que frequência acontecem os retornos extremos?	2
Processos estocásticos	Que modelo de dinâmica gera os preços observados?	3
Leis de potência e escala	As caudas e as flutuações seguem leis de escala?	4
Fractais e multifractais	Os retornos têm memória longa? Como ela escala no tempo?	5
Matrizes aleatórias e redes	Quanto das correlações entre ativos é sinal, e quanto é ruído?	6
Entropia e informação	Quão previsível é uma série? Quem influencia quem?	7
Modelos de agentes	Que interações entre investidores geram os fatos observados?	8
Pontos críticos e extremos	Como medir o risco de crises? Bolhas têm assinatura?	9

Três atitudes atravessam todos esses métodos. A primeira é o **empirismo**: o ponto de partida são as regularidades medidas nos dados, e não hipóteses sobre o comportamento dos investidores. A segunda é a busca de **modelos mínimos**: o modelo mais simples que reproduz um fenômeno ensina mais do que um modelo com muitos parâmetros ajustados. A terceira é a **simulação**: quando um modelo não tem solução analítica, ele é explorado no computador, como você fez com o passeio aleatório no Caderno 0.

1.4 Econofísica, economia e finanças

A Econofísica não substitui a economia nem as finanças. A teoria econômica pergunta *por que* os agentes agem como agem e qual é o equilíbrio resultante; as finanças desenvolveram a teoria do apreçamento de ativos e da gestão de carteiras; a econometria financeira construiu modelos estatísticos rigorosos para séries de preços. A Econofísica contribui com outra pergunta: *que leis estatísticas os preços obedecem*, e que mecanismos coletivos podem produzi-las.

As duas abordagens frequentemente estudam os mesmos fenômenos. O agrupamento de volatilidade, por exemplo, foi medido por econofísicos com expoentes de escala e por econometristas com modelos da família GARCH (Capítulo 5). Este curso trata a Econofísica como complemento: sempre que possível, os resultados serão comparados com os da abordagem clássica.

Contexto de mercado

O resto do caderno ilustra essa relação. O Capítulo 2 mostra que a ideia de passeio aleatório veio de um matemático e de um físico; o Capítulo 3, que ela virou o centro da teoria financeira; e o Capítulo 4, onde ela falha, no que os dados mostram.

1.5 Os dados deste caderno

Este caderno usa duas séries longas de índices de ações: o S&P 500, desde 1950, e o Ibovespa, desde 2000. Períodos longos são necessários para estudar eventos raros, como a queda de 1987, e propriedades que só aparecem em amostras grandes, como a forma das caudas. A Listagem 1.1 baixa as duas séries do Yahoo Finance e grava cópias datadas, como no Caderno 0.

Listagem 1.1 Baixar as séries longas do S&P 500 e do Ibovespa.

```python
from datetime import date
from pathlib import Path

import yfinance as yf

# Séries longas: S&P 500 desde 1950 e Ibovespa desde 2000
series = {"sp500": ("^GSPC", "1950-01-01"),
          "ibovespa": ("^BVSP", "2000-01-01")}
for nome, (codigo, inicio) in series.items():
    precos = yf.download(codigo, start=inicio, end="2026-01-01",
                         auto_adjust=True, progress=False)["Close"]
    arquivo = Path("dados") / f"yahoo_{nome}_{date.today()}.csv"
    precos.to_csv(arquivo)
    print(arquivo, precos.shape)
```

```
dados/yahoo_sp500_2026-10-10.csv (19121, 1)
dados/yahoo_ibovespa_2026-10-10.csv (6441, 1)
```

Como todos os capítulos precisam dos mesmos log-retornos, as funções que leem os arquivos ficam num módulo, `dados_caderno1.py` (Listagem 1.2). Cada capítulo só precisa importá-lo.

Listagem 1.2 O módulo de dados do caderno.

```python
"""Funções de dados usadas em todos os capítulos do Caderno 1."""
from pathlib import Path

import numpy as np
import pandas as pd


def arquivo_mais_recente(padrao):
    """Devolve a cópia mais recente de um arquivo (nomes com data ISO)."""
    copias = sorted(Path("dados").glob(padrao))
    if not copias:
        raise FileNotFoundError(f"Sem arquivo {padrao}: rode a Listagem 1.1")
    return copias[-1]


def log_retornos(precos):
    """Log-retornos de uma série de preços, só nos dias com cotação."""
    return np.log(precos.dropna()).diff().dropna()


def carregar_retornos():
    """Log-retornos diários do S&P 500 e do Ibovespa."""
    sp500 = pd.read_csv(arquivo_mais_recente("yahoo_sp500_*.csv"),
                        index_col="Date", parse_dates=True)["^GSPC"]
    ibov = pd.read_csv(arquivo_mais_recente("yahoo_ibovespa_*.csv"),
                       index_col="Date", parse_dates=True)["^BVSP"]
    return log_retornos(sp500), log_retornos(ibov)
```

Boa prática

Quando o mesmo código aparece em mais de um notebook, mova-o para um arquivo `.py` e importe-o. Uma correção feita no módulo vale para todos os notebooks, e nenhum deles fica com uma versão antiga da função.

Listagem 1.3 Carregar os log-retornos com o módulo.

```python
from dados_caderno1 import carregar_retornos

r_sp, r_ibov = carregar_retornos()
for nome, r in [("S&P 500", r_sp), ("Ibovespa", r_ibov)]:
    print(f"{nome:<9} {len(r):6d} retornos, de {r.index.min().date()} "
          f"a {r.index.max().date()}")
```

```
S&P 500    19120 retornos, de 1950-01-04 a 2025-12-31
Ibovespa    6440 retornos, de 2000-01-04 a 2025-12-30
```

São 19 120 retornos diários do S&P 500 e 6 440 do Ibovespa. Antes de usar os dados, um diagnóstico rápido: quantos retornos são exatamente iguais a zero, isto é, dias em que o preço de fechamento repetiu o do dia anterior?

Listagem 1.4 Retornos iguais a zero, por década.

```python
# Retornos exatamente iguais a zero: preço repetido de um dia para o outro
zeros = r_sp[r_sp == 0]
print(zeros.groupby(zeros.index.year // 10 * 10).size())
print("Ibovespa:", (r_ibov == 0).sum(), "retornos iguais a zero")
```

```
Date
1950    67
1960    29
1970    18
1980     6
1990     2
2000     2
2010     1
Name: ^GSPC, dtype: int64
Ibovespa: 4 retornos iguais a zero
```

Os zeros do S&P 500 se concentram nos anos 1950 e 1960. Naquela época, o índice valia algumas dezenas de pontos, e a série tem só duas casas decimais: o mais provável é que variações menores que um centésimo de ponto tenham virado zero no arredondamento. São poucos (125 em mais de 19 mil dias) e não afetam as conclusões do caderno, mas é o tipo de detalhe que se documenta antes de começar.

Resumo do capítulo

- A Econofísica aplica métodos da física estatística e dos sistemas complexos à economia e às finanças.
- Os mercados atraíram físicos por terem dados abundantes, fenômenos coletivos e leis de escala.
- Os métodos (caudas pesadas, processos estocásticos, escala, fractais, matrizes aleatórias, entropia, agentes, extremos) organizam os módulos do curso.
- Empirismo, modelos mínimos e simulação são as atitudes da área.
- A Econofísica complementa a economia, as finanças e a econometria; não as substitui.
- Os dados do caderno: S&P 500 (1950–2025) e Ibovespa (2000–2025), lidos por um módulo próprio.

Exercícios

Essencial — fixação

Exercício 1.1 Execute as Listagens 1.1 a 1.4. Em que ano o S&P 500 teve mais retornos iguais a zero?

Exercício 1.2 Para cada linha da Tabela 1.1, dê um exemplo de pergunta prática que um analista de risco ou um gestor de carteira faria.

Aplicado — o mesmo método num problema novo

Exercício 1.3 Acrescente ao módulo `dados_caderno1.py` uma função `carregar_precos()` que devolva os preços (e não os retornos) das duas séries, e use-a para calcular a variação total de cada índice no período.

Desafio — extensão aberta

Exercício 1.4 Procure um fenômeno de mercado que lhe pareça “coletivo” (uma corrida a um banco, uma bolha, um pânico) e descreva, em um parágrafo, que tipo de interação entre participantes poderia produzi-lo. Guarde a resposta para o Módulo 8.

CAPÍTULO 2

De Bachelier a Mandelbrot

◎ Objetivos do capítulo

- Conhecer o passeio aleatório de Bachelier e a lei da raiz do tempo.
- Entender por que Osborne passou a modelar o logaritmo do preço.
- Reconhecer a crítica de Mandelbrot ao modelo gaussiano.
- Medir, com dados, quão improvável a queda de 1987 seria sob a normal.

A ideia de que os preços se movem ao acaso tem mais de um século. Este capítulo conta a sua história em quatro momentos e, em cada um, reproduz no computador o argumento central.

2.1 Bachelier e o passeio aleatório (1900)

Em março de 1900, Louis Bachelier defendeu em Paris a tese *Théorie de la spéculation*, orientada por Henri Poincaré (BACHELIER, 1900). Bachelier estudava os preços dos títulos da dívida pública francesa e das opções sobre eles. A sua hipótese central: as variações de preço são a soma de muitos pequenos choques independentes, de média zero. O preço faz um *passeio aleatório*.

Para descrever esse movimento, Bachelier desenvolveu a matemática do que hoje se chama *movimento browniano*, cinco anos antes do famoso artigo de Einstein sobre o movimento de partículas num fluido. A tese, porém, permaneceu pouco conhecida entre os economistas até ser redescoberta nos anos 1950.

A principal consequência do modelo é a *lei da raiz do tempo*: se cada dia soma um choque independente de desvio-padrão σ, a dispersão do preço depois de t dias é $\sigma\sqrt{t}$, e não σt. Para quadruplicar a incerteza, é preciso esperar dezesseis vezes mais. A Listagem 2.1 confere a lei com 10 000 trajetórias simuladas.

Listagem 2.1 O passeio aleatório de Bachelier e a lei da raiz do tempo.

```
import numpy as np

# Passeio aleatório aritmético de Bachelier: o preço soma choques normais
rng = np.random.default_rng(seed=42)
dias, trajetorias, sigma = 400, 10_000, 1.0     # sigma em R$ por dia
choques = rng.normal(0, sigma, size=(dias, trajetorias))
precos = 100 + np.cumsum(choques, axis=0)
```

```

for t in [25, 100, 400]:
    observado = precos[t - 1].std()
    print(f"t = {t:3d} dias: desvio {observado:5.2f}   sigma*raiz(t) = "
          f"{sigma * np.sqrt(t):5.2f}")
```

```
t =  25 dias: desvio  4.95   sigma*raiz(t) =  5.00
t = 100 dias: desvio 10.02   sigma*raiz(t) = 10.00
t = 400 dias: desvio 20.12   sigma*raiz(t) = 20.00
```

Com 25, 100 e 400 dias, o desvio observado é 5, 10 e 20: cada vez que o horizonte quadruplica, a dispersão dobra. Você já usou essa lei no Caderno 0, ao anualizar a volatilidade com $\sqrt{252}$.

2.2 Osborne e o movimento browniano geométrico (1959)

O modelo de Bachelier tem um defeito: como soma choques em reais, e não em porcentagem, o preço pode ficar negativo. Em 1959, o físico M. F. M. Osborne, analisando ações norte-americanas, propôs que o passeio aleatório se aplica ao *logaritmo* do preço (OSBORNE, 1959). As variações passam a ser proporcionais ao preço, e o preço nunca cruza o zero. É o *movimento browniano geométrico*, o mesmo modelo que você simulou no Capítulo 3 do Caderno 0.

Listagem 2.2 Passeio aritmético contra passeio geométrico em 25 anos.

```
import matplotlib.pyplot as plt

# Mesma volatilidade inicial, 25 anos: aritmético (Bachelier) e
# geométrico (Osborne)
rng = np.random.default_rng(seed=7)
dias, n = 25 * 252, 20
choques = rng.normal(0, 0.02, size=(dias, n))       # 2% ao dia
aritmetico = 100 + np.cumsum(100 * choques, axis=0)
geometrico = 100 * np.exp(np.cumsum(choques, axis=0))

print("Trajetórias que ficaram negativas, de", n)
print("  aritmético:", (aritmetico.min(axis=0) < 0).sum())
print("  geométrico:", (geometrico.min(axis=0) < 0).sum())

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(6.4, 2.9), sharey=True)
anos = np.arange(dias) / 252
ax1.plot(anos, aritmetico, linewidth=0.6)
ax1.axhline(0, color="black", linewidth=0.8)
ax1.set_title("Aritmético (Bachelier)")
ax2.plot(anos, geometrico, linewidth=0.6)
ax2.axhline(0, color="black", linewidth=0.8)
ax2.set_title("Geométrico (Osborne)")
for ax in (ax1, ax2):
    ax.set_xlabel("anos")
ax1.set_ylabel("preço")
fig.savefig("figuras/fig_02_01_bachelier_osborne.pdf")
```

```
Trajetórias que ficaram negativas, de 20
  aritmético: 10
  geométrico: 0
```

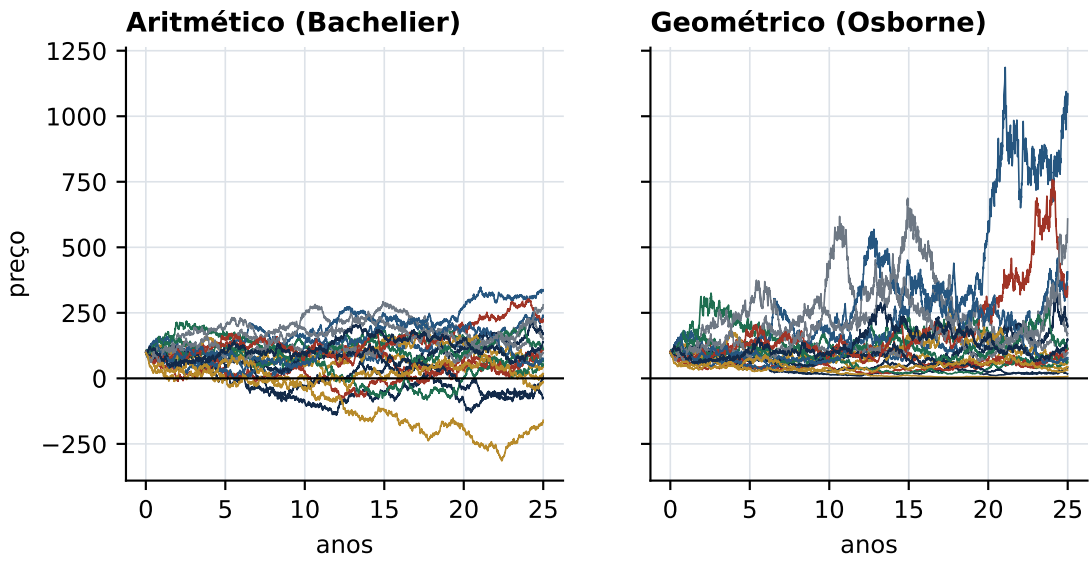


Figura 2.1 Vinte trajetórias de cada modelo ao longo de 25 anos, com choques de 2% ao dia. No passeio aritmético, metade das trajetórias passou por preços negativos; no geométrico, nenhuma.

Nota

O movimento browniano geométrico é a hipótese sobre os preços do modelo de Black e Scholes (BLACK; SCHOLES, 1973) para o preço de opções, um dos resultados mais influentes das finanças. É por isso que o modelo gaussiano de retornos logarítmicos virou a referência de toda a área (Seção 4.1).

2.3 Mandelbrot e as caudas pesadas (1963)

Em 1963, Benoit Mandelbrot publicou um estudo das variações do preço do algodão nos Estados Unidos (MANDELBROT, 1963). A conclusão contrariava o modelo gaussiano: variações extremas eram muito mais frequentes do que a normal permite, e a distribuição dos preços tinha *caudas pesadas*. Mandelbrot propôs substituí-la pelas distribuições *estáveis de Lévy*, que têm caudas em lei de potência e, em certos casos, variância infinita. Pouco depois, a tese de Eugene Fama encontrou o mesmo excesso de valores extremos nas ações do índice Dow Jones (FAMA, 1965).

A Listagem 2.3 mostra a força do argumento de Mandelbrot com o evento mais extremo da série do S&P 500: a queda de 19 de outubro de 1987, a *Segunda-Feira Negra*. A pergunta é: com tudo o que se sabia até a véspera, quão improvável essa queda parecia para o modelo gaussiano?

Listagem 2.3 A queda de 1987 medida em desvios-padrão.

```
import pandas as pd
from scipy import stats

from dados_caderno1 import carregar_retornos

r_sp, _ = carregar_retornos()
historico = r_sp.loc[:"1987-10-16"]      # o que se sabia na véspera
```

```
queda = r_sp.loc["1987-10-19"]

z = (queda - historico.mean()) / historico.std()
print(f"Log-retorno em 19/10/1987: {queda:.1%}")
print(f"Desvio-padrão diário de 1950 até a véspera: {historico.std():.2%}")
print(f"A queda equivale a {z:.1f} desvios-padrão")
print(f"Probabilidade pela normal: {stats.norm.cdf(z):.1e}")
```

```
Log-retorno em 19/10/1987: -22.9%
Desvio-padrão diário de 1950 até a véspera: 0.79%
A queda equivale a -29.1 desvios-padrão
Probabilidade pela normal: 7.3e-187
```

O log-retorno foi de $-22{,}9\%$, ou 29 desvios-padrão abaixo da média histórica. Pela normal, a probabilidade de uma queda assim num dia qualquer é da ordem de 10^{-187}. Para comparação, a idade do universo é de cerca de 5×10^{12} dias. Sob a normal, a Segunda-Feira Negra simplesmente não poderia ter acontecido. Ela aconteceu, e o modelo é que precisa ser revisto.

Armadilha comum

Chamar eventos assim de "inesperados" ou de "eventos de 29 sigmas" como se fossem azar. Um evento de 29 desvios-padrão não diz que o mercado teve um dia de muito azar: diz que a distribuição normal é o modelo errado para as caudas. O Módulo 2 mostra como medir as caudas de verdade.

2.4 A física estatística chega aos mercados (anos 1990)

As ideias de Mandelbrot não foram adotadas pela corrente principal das finanças, em parte porque as distribuições estáveis tornam muitos cálculos difíceis. Nas décadas seguintes, a teoria financeira se construiu sobre o modelo gaussiano. A Figura 2.2 mostra o que os dados continuavam registrando enquanto isso.

Listagem 2.4 Os log-retornos diários do S&P 500 desde 1950.

```
fig, ax = plt.subplots(figsize=(6.4, 3.0))
ax.plot(r_sp.index, r_sp, linewidth=0.4, color="C0")
for data, rotulo in [("1987-10-19", "1987"), ("2008-10-15", "2008"),
                     ("2020-03-16", "2020")]:
    ax.annotate(rotulo, (pd.Timestamp(data), r_sp.loc[data]),
                xytext=(6, -2), textcoords="offset points", fontsize=8,
                color="C3")
ax.set_title("S&P 500: log-retornos diários, 1950 a 2025")
fig.savefig("figuras/fig_02_02_sp500_historia.pdf")
print(r_sp.nsmallest(3).round(3))
```

```
Date
1987-10-19   -0.229
2020-03-16   -0.128
2020-03-12   -0.100
Name: ^GSPC, dtype: float64
```

Nos anos 1990, com séries de alta frequência disponíveis e computadores capazes de processá-las, físicos começaram a estudar esses dados com as ferramentas da física estatística. Um marco é o artigo de Mantegna e Stanley de 1995 na revista *Nature*, que analisou as

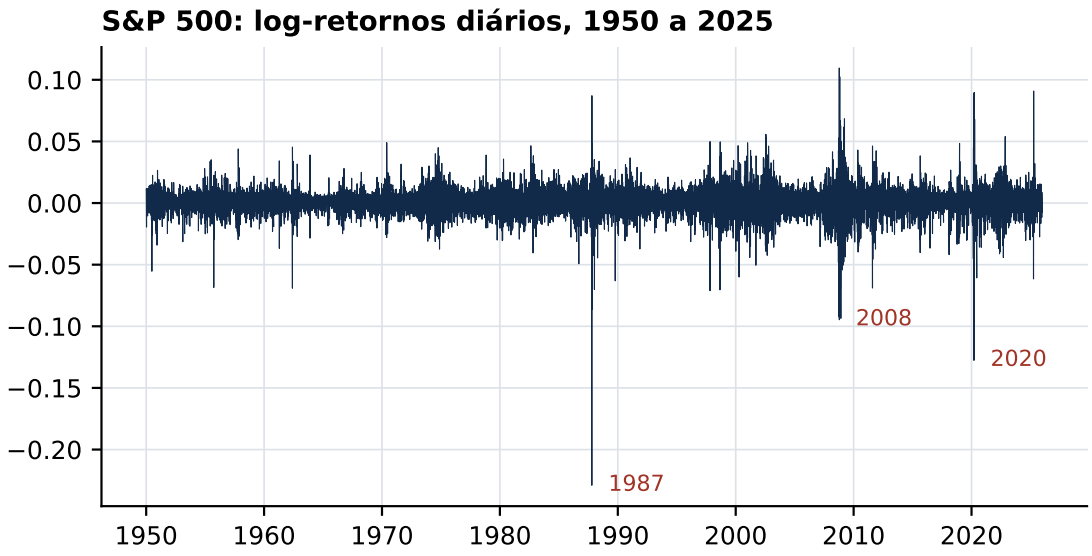


Figura 2.2 Log-retornos diários do S&P 500 de 1950 a 2025. Os três piores dias da série são o de 1987 e dois de março de 2020.

variações do S&P 500 em diferentes escalas de tempo e encontrou leis de escala na sua distribuição (MANTEGNA; STANLEY, 1995). A Econofísica nasceu desse encontro entre a crítica de Mandelbrot, dados abundantes e os métodos da física dos sistemas complexos.

Contexto de mercado

Repare na Figura 2.2: os retornos grandes não estão espalhados ao acaso ao longo dos 75 anos. Eles se agrupam em alguns períodos, como 1987, 2008 e 2020. Esse agrupamento de volatilidade é o tema do Capítulo 4 e, com ferramentas mais finas, do Módulo 5.

Resumo do capítulo

- Bachelier (1900): o preço como passeio aleatório; a dispersão cresce como $\sigma\sqrt{t}$.
- Osborne (1959): o passeio se aplica ao logaritmo do preço; o movimento browniano geométrico nunca fica negativo e é a base do modelo de Black e Scholes.
- Mandelbrot (1963) e Fama (1965): as variações de preço têm caudas pesadas, incompatíveis com a normal.
- A queda de 1987 foi de 29 desvios-padrão: sob a normal, um evento impossível.
- Anos 1990: a física estatística e os dados abundantes dão origem à Econofísica.

Exercícios

Essencial — fixação

Exercício 2.1 Na Listagem 2.1, calcule também o desvio no dia 900. Qual valor a lei da raiz do tempo prevê?

Exercício 2.2 Repita a Listagem 2.3 para o pior dia de 2008 e o pior de 2020 do S&P 500, usando em cada caso o histórico até a véspera. Quantos desvios-padrão foram?

Aplicado — o mesmo método num problema novo

Exercício 2.3 Faça o mesmo cálculo para a pior queda do Ibovespa entre 2000 e 2025. Ela é mais ou menos extrema, em desvios-padrão, do que a do S&P 500 em 1987?

Exercício 2.4 Na Listagem 2.2, aumente o número de trajetórias para 1 000 e estime a fração de trajetórias aritméticas que ficam negativas em 5, 10 e 25 anos.

Desafio — extensão aberta

Exercício 2.5 Usando a `stats.t` ajustada aos retornos do S&P 500 até a véspera de 19/10/1987, calcule a probabilidade da queda daquele dia. A t de Student torna o evento plausível? O que isso sugere sobre a forma das caudas?

CAPÍTULO 3

Mercados eficientes e passeio aleatório

◎ Objetivos do capítulo

- Enunciar a hipótese dos mercados eficientes e as suas três formas.
- Relacionar eficiência, passeio aleatório e ausência de autocorrelação.
- Calcular autocorrelações, construir um correlograma e aplicar o teste de Ljung-Box.
- Usar a razão de variâncias, com uma referência obtida por embaralhamento.

3.1 A hipótese dos mercados eficientes

O passeio aleatório de Bachelier ganhou uma justificativa econômica nos anos 1960. Em 1965, Paul Samuelson mostrou que, num mercado em que os participantes usam toda a informação disponível, as variações de preço bem antecipadas devem ser imprevisíveis (SAMUELSON, 1965): se houvesse um padrão previsível, os investidores o explorariam até ele desaparecer. Em 1970, Eugene Fama sistematizou a *hipótese dos mercados eficientes*: um mercado é eficiente quando os preços refletem plenamente a informação disponível (FAMA, 1970).

Fama distinguiu três formas da hipótese, conforme a informação considerada:

- **fraca**: os preços refletem toda a informação contida nos preços passados;
- **semiforte**: refletem toda a informação pública (balanços, notícias, anúncios);
- **forte**: refletem toda a informação, inclusive a privilegiada.

Este capítulo testa a forma fraca, a única que se pode examinar só com séries de preços. Se ela vale, os retornos passados não ajudam a prever os retornos futuros.

Nota

O debate sobre a eficiência dos mercados continua aberto. Em 2013, o Prêmio Nobel de Economia foi dividido entre Eugene Fama, um dos formuladores da hipótese, e Robert Shiller, um dos seus principais críticos, além de Lars Peter Hansen, pela análise empírica dos preços de ativos.

3.2 Do passeio aleatório à ausência de autocorrelação

Se os retornos são imprevisíveis a partir do passado, a correlação entre o retorno de hoje e o de k dias atrás deve ser nula. Essa correlação de uma série com ela mesma, defasada, é a *autocorrelação*:

$$\rho(k) = \frac{\sum_t (r_t - \bar{r})(r_{t-k} - \bar{r})}{\sum_t (r_t - \bar{r})^2}.$$

A Listagem 3.2 implementa a fórmula com NumPy, depois de carregar os dados.

Listagem 3.1 Dados do capítulo.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from dados_caderno1 import carregar_retornos

r_sp, r_ibov = carregar_retornos()
```

Listagem 3.2 A autocorrelação calculada à mão.

```
def autocorrelacao(x, k):
    """Correlação entre a série e ela mesma k passos antes."""
    x = np.asarray(x) - np.mean(x)
    return np.sum(x[k:] * x[:-k]) / np.sum(x * x)


for nome, serie in [("retornos", r_ibov), ("absolutos", r_ibov.abs())]:
    valores = np.array([autocorrelacao(serie, k) for k in range(1, 6)])
    print(f"Ibovespa, {nome:<9} (defasagens 1 a 5): {valores.round(3)}")
```

```
Ibovespa, retornos  (defasagens 1 a 5): [-0.024 -0.005 -0.03  -0.009  0.003]
Ibovespa, absolutos (defasagens 1 a 5): [0.208 0.277 0.246 0.227 0.233]
```

As autocorrelações dos retornos do Ibovespa são minúsculas: nenhuma passa de 0,03 em valor absoluto. As dos *retornos absolutos*, que medem o tamanho do movimento sem o sinal, são dez vezes maiores, entre 0,2 e 0,3. A direção do próximo movimento é quase imprevisível; o tamanho, não.

3.3 Testando: correlograma e Ljung-Box

Na prática, usa-se a função `acf` da biblioteca `statsmodels`, que faz a mesma conta. Para decidir se um conjunto de autocorrelações é compatível com zero, o *teste de Ljung-Box* combina as m primeiras numa só estatística,

$$Q = n(n+2) \sum_{k=1}^{m} \frac{\rho(k)^2}{n-k},$$

que, sem autocorrelação, tem distribuição qui-quadrado com m graus de liberdade (LJUNG; BOX, 1978).

Listagem 3.3 A mesma conta com o `statsmodels` e o teste de Ljung-Box.

```python
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.tsa.stattools import acf

# A função acf do statsmodels faz a mesma conta da Listagem 3.2
print(np.allclose(acf(r_ibov, nlags=5)[1:],
                  [autocorrelacao(r_ibov, k) for k in range(1, 6)]))

# Ljung-Box: as 10 primeiras autocorrelações são todas nulas?
for nome, serie in [("retornos", r_ibov), ("absolutos", r_ibov.abs())]:
    teste = acorr_ljungbox(serie, lags=[10])
    print(f"{nome:<10} Q = {teste['lb_stat'].iloc[0]:8.1f}   "
          f"p = {teste['lb_pvalue'].iloc[0]:.2g}")
```

```
True
retornos   Q =     24.4   p = 0.0067
absolutos  Q =   3253.2   p = 0
```

O teste rejeita a ausência de autocorrelação nas duas séries, mas com intensidades muito diferentes. Para os retornos, $Q = 24$ e $p = 0{,}007$: há alguma autocorrelação, estatisticamente detectável num sinal de 6 440 dias, mas pequena demais para ser explorada depois de custos de transação. Para os retornos absolutos, Q passa de três mil, e o valor-p é zero na precisão do computador.

O *correlograma* mostra as autocorrelações para muitas defasagens, com a faixa $\pm 1{,}96/\sqrt{n}$, dentro da qual ficam 95% das autocorrelações de uma série sem memória.

Listagem 3.4 Correlogramas dos retornos e dos retornos absolutos.

```python
defasagens = np.arange(1, 101)
limite = 1.96 / np.sqrt(len(r_ibov))      # faixa de 95% sem correlação

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(6.4, 4.2), sharex=True)
for ax, serie, titulo in [(ax1, r_ibov, "Retornos"),
                          (ax2, r_ibov.abs(), "Retornos absolutos")]:
    ax.bar(defasagens, acf(serie, nlags=100)[1:], width=0.8)
    ax.axhspan(-limite, limite, color="C1", alpha=0.25)
    ax.set_title(f"{titulo} do Ibovespa: autocorrelação")
ax2.set_xlabel("defasagem (pregões)")
fig.savefig("figuras/fig_03_01_correlograma.pdf")
print(f"faixa de 95%: +/- {limite:.3f}")
```

```
faixa de 95%: +/- 0.024
```

A Figura 3.1 resume a situação do passeio aleatório. Para os retornos, o modelo é uma boa aproximação. Para os retornos absolutos, falha completamente: a autocorrelação ainda é visível depois de 100 pregões, cerca de cinco meses.

Contexto de mercado

Retornos sem autocorrelação e retornos absolutos com autocorrelação longa são dois dos fatos estilizados mais robustos (CONT, 2001). Juntos, dizem que o mercado é eficiente quanto à direção dos preços, mas não quanto à intensidade das variações. Ding, Granger e Engle documentaram essa memória longa nos retornos absolutos do S&P 500 (DING; GRANGER; ENGLE, 1993); o Módulo 5 a mede com o expoente de Hurst.

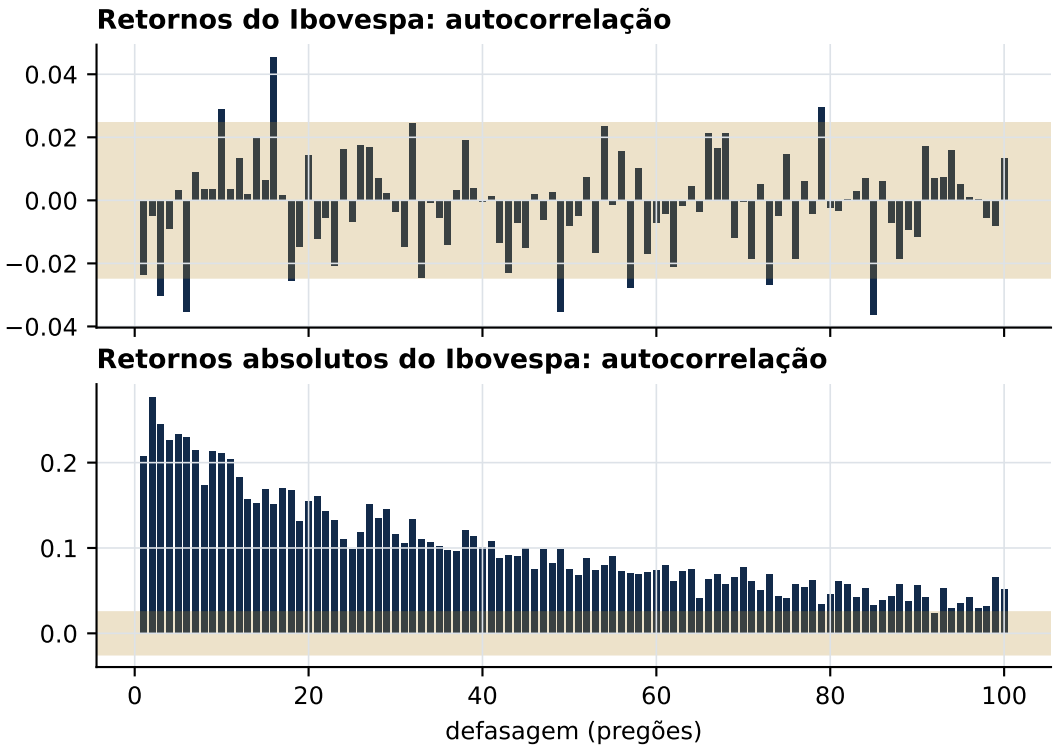


Figura 3.1 Autocorrelações do Ibovespa até 100 pregões. Acima, os retornos oscilam em torno da faixa de 95% (em amarelo). Abaixo, os retornos absolutos ficam muito acima da faixa e decaem devagar.

3.4 A eficiência muda com o tempo

Uma série longa permite perguntar se a autocorrelação é estável. A Listagem 3.5 calcula a autocorrelação de um pregão do S&P 500 em quatro períodos de cerca de 20 anos.

Listagem 3.5 Autocorrelação de um pregão do S&P 500 por período.

```
# Autocorrelação de 1 pregão do S&P 500, por período
for inicio, fim in [("1950", "1969"), ("1970", "1989"), ("1990", "2009"),
                    ("2010", "2025")]:
    trecho = r_sp.loc[inicio:fim]
    print(f"{inicio}–{fim}: {trecho.autocorr(1):+.3f}  "
          f"(faixa de 95%: +/- {1.96 / np.sqrt(len(trecho)):.3f})")
```

```
1950–1969: +0.120  (faixa de 95%: +/- 0.028)
1970–1989: +0.127  (faixa de 95%: +/- 0.028)
1990–2009: −0.056  (faixa de 95%: +/- 0.028)
2010–2025: −0.114  (faixa de 95%: +/- 0.031)
```

O resultado é surpreendente: a autocorrelação foi positiva e bem fora da faixa de 95% entre 1950 e 1989 e passou a ser negativa desde 1990. Uma das explicações discutidas na literatura para a autocorrelação positiva antiga de índices é a negociação não sincrônica: nem todas as ações do índice negociavam até o fim de cada pregão, e o índice incorporava as notícias com atraso. Seja qual for a causa, o exemplo mostra que a "eficiência" não é uma propriedade fixa de um mercado: muda com a tecnologia, os participantes e as regras.

3.5 A razão de variâncias

Outra forma de testar o passeio aleatório usa a lei da raiz do tempo do Capítulo 2. Se os retornos são independentes, a variância do retorno de q pregões é q vezes a variância diária, e a *razão de variâncias*

$$\mathrm{VR}(q) = \frac{\mathrm{Var}(r_t + r_{t-1} + \cdots + r_{t-q+1})}{q\,\mathrm{Var}(r_t)}$$

vale 1. Valores acima de 1 indicam autocorrelação positiva (tendência); abaixo de 1, negativa (reversão). Lo e MacKinlay usaram essa estatística para rejeitar o passeio aleatório nos retornos semanais de índices norte-americanos entre 1962 e 1985 (LO; MacKINLAY, 1988).

Em vez da distribuição teórica da estatística, a Listagem 3.6 usa uma referência típica da Econofísica: *embaralhar* a série. Embaralhar mantém exatamente a mesma distribuição dos retornos e destrói qualquer dependência no tempo. A faixa entre os percentis 2,5% e 97,5% de 200 embaralhamentos mostra o que se espera de uma série sem memória.

Listagem 3.6 Razão de variâncias e a referência por embaralhamento.

```
def razao_variancias(r, q):
    """Var(retorno de q pregões) / (q * Var(retorno de 1 pregão))."""
    x = np.asarray(r)
    n = len(x) // q
    somas = x[:n * q].reshape(n, q).sum(axis=1)   # blocos sem sobreposição
    return np.var(somas, ddof=1) / (q * np.var(x, ddof=1))


rng = np.random.default_rng(seed=42)
for nome, r in [("S&P 500", r_sp), ("Ibovespa", r_ibov)]:
    print(nome)
    for q in [2, 5, 10, 20]:
        # Referência: a mesma série embaralhada não tem memória nenhuma
        nulas = [razao_variancias(rng.permutation(r.values), q)
                 for _ in range(200)]
        baixo, alto = np.percentile(nulas, [2.5, 97.5])
        print(f"  q = {q:2d}: {razao_variancias(r, q):.3f}    "
              f"embaralhada: {baixo:.3f} a {alto:.3f}")
```

```
S&P 500
  q =  2: 0.971   embaralhada: 0.978 a 1.019
  q =  5: 0.945   embaralhada: 0.961 a 1.051
  q = 10: 0.891   embaralhada: 0.943 a 1.061
  q = 20: 0.936   embaralhada: 0.925 a 1.099
Ibovespa
  q =  2: 0.994   embaralhada: 0.970 a 1.032
  q =  5: 0.903   embaralhada: 0.932 a 1.076
  q = 10: 0.884   embaralhada: 0.910 a 1.080
  q = 20: 0.965   embaralhada: 0.866 a 1.153
```

Para $q = 5$ e $q = 10$, isto é, de uma a duas semanas, as duas séries ficam abaixo da faixa: há uma leve reversão nesse horizonte. Para $q = 2$ e $q = 20$, os valores ficam dentro da faixa, ou muito perto dela. Os desvios do passeio aleatório nos retornos existem, mas são pequenos, dependem do horizonte e do período, como já indicava a Listagem 3.5.

Boa prática

Quando a distribuição teórica de uma estatística é difícil de obter, ou depende de hipóteses que os dados violam (como a normalidade), compare o valor observado com o de séries embaralhadas ou simuladas. O embaralhamento responde a uma pergunta precisa: este resultado poderia aparecer numa série com a mesma distribuição e sem memória nenhuma?

Resumo do capítulo

- Hipótese dos mercados eficientes (Fama, 1970): os preços refletem a informação disponível; formas fraca, semiforte e forte.
- Forma fraca $\Rightarrow$ retornos imprevisíveis a partir do passado $\Rightarrow$ autocorrelações nulas.
- Nos dados: autocorrelação dos retornos pequena (estatisticamente detectável em séries longas); dos retornos absolutos, grande e persistente.
- Ljung-Box testa várias autocorrelações ao mesmo tempo; o correlograma as mostra com a faixa $\pm 1{,}96/\sqrt{n}$.
- A autocorrelação do S&P 500 mudou de sinal ao longo das décadas.
- A razão de variâncias compara horizontes; séries embaralhadas dão a referência sem memória.

Exercícios

Essencial — fixação

Exercício 3.1 Use a função `autocorrelacao` da Listagem 3.2 nos retornos do S&P 500 e nos seus valores absolutos. Os resultados se parecem com os do Ibovespa?

Exercício 3.2 Aplique o teste de Ljung-Box aos retornos do Ibovespa com 5, 20 e 50 defasagens. A conclusão depende do número de defasagens?

Aplicado — o mesmo método num problema novo

Exercício 3.3 Refaça a Listagem 3.5 para o Ibovespa, com períodos de 5 anos. A autocorrelação de um pregão também mudou de sinal?

Exercício 3.4 Calcule a razão de variâncias dos *retornos absolutos* do Ibovespa para $q = 5, 10, 20$, com a referência por embaralhamento. Interprete o resultado à luz da Figura 3.1.

Desafio — extensão aberta

Exercício 3.5 Uma estratégia ingênua compra o índice no dia seguinte a uma alta e vende no dia seguinte a uma queda. Simule o seu retorno acumulado no S&P 500 em cada um dos quatro períodos da Listagem 3.5, sem custos de transação. O resultado acompanha o sinal da autocorrelação? O que acontece com custos de 0,1% por operação?

CAPÍTULO 4

O modelo gaussiano e os fatos estilizados

◎ Objetivos do capítulo

- Explicar por que a distribuição normal é o modelo de referência para retornos.
- Conhecer os fatos estilizados dos retornos financeiros.
- Comparar um índice real com uma amostra gaussiana de mesma média e variância (prática principal do módulo).
- Medir a gaussianidade agregacional e as assimetrias entre ganhos e perdas.

4.1 Por que a normal é a referência

O modelo de Bachelier e o de Osborne supõem que os retornos são normais. A escolha não é arbitrária. O *teorema central do limite* garante que a soma de muitos choques independentes, de variância finita, se aproxima da distribuição normal, qualquer que seja a distribuição de cada choque. Se o retorno de um dia resulta de muitas pequenas decisões independentes, ele deve ser aproximadamente normal.

Listagem 4.1 Dados do capítulo.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy import stats
from statsmodels.tsa.stattools import acf

from dados_caderno1 import carregar_retornos

r_sp, r_ibov = carregar_retornos()
```

Listagem 4.2 O teorema central do limite com choques uniformes.

```
# Teorema central do limite: somar muitos choques independentes
rng = np.random.default_rng(seed=42)
for n in [1, 2, 10, 50]:
    soma = rng.uniform(-1, 1, size=(100_000, n)).sum(axis=1)
    print(f"soma de {n:2d} choques uniformes: curtose em excesso "
```

```
            f"{stats.kurtosis(soma):+.3f}")
```

```
soma de  1 choques uniformes: curtose em excesso -1.197
soma de  2 choques uniformes: curtose em excesso -0.592
soma de 10 choques uniformes: curtose em excesso -0.134
soma de 50 choques uniformes: curtose em excesso -0.039
```

Um choque uniforme tem curtose em excesso de $-1{,}2$: caudas mais *leves* que as da normal. Somando 50 deles, a curtose em excesso fica praticamente em zero, o valor da normal. Além dessa justificativa, a normal tem vantagens práticas: é descrita por só dois parâmetros, média e desvio-padrão, e a soma de variáveis normais independentes continua normal, o que torna muitos cálculos de risco e de apreçamento exatos.

O problema é a hipótese do teorema: choques *independentes* e de variância *finita*. Se os choques forem dependentes, ou se as suas caudas forem pesadas demais, o teorema não garante a convergência para a normal, e os dados dizem que é isso que acontece.

4.2 Os fatos estilizados

Um *fato estilizado* é uma regularidade estatística observada de forma consistente em muitos ativos, mercados e períodos. Rama Cont reuniu os principais numa lista que virou referência (CONT, 2001). A Tabela 4.1 os resume e indica onde cada um é tratado.

Tabela 4.1 Os fatos estilizados dos retornos (CONT, 2001) e onde são tratados no curso.

Fato	O que diz	Onde
Ausência de autocorrelação	Os retornos quase não têm autocorrelação linear, salvo em escalas de tempo muito curtas	C1, cap. 3
Caudas pesadas	Retornos extremos são muito mais frequentes do que na normal	C1; M2
Assimetria ganho-perda	Quedas extremas são maiores ou mais numerosas que altas extremas	C1, cap. 4
Gaussianidade agregacional	A distribuição se aproxima da normal em horizontes longos	C1, cap. 4
Intermitência	Os retornos alternam surtos de variações grandes com períodos calmos	C1; M5
Agrupamento de volatilidade	Retornos grandes tendem a ser seguidos por retornos grandes, de qualquer sinal	C1; M5
Caudas pesadas condicionais	Mesmo descontada a volatilidade variável, as caudas continuam pesadas	M2, M9
Decaimento lento da autocorrelação dos absolutos	A autocorrelação dos retornos absolutos decai devagar, como uma lei de potência	C1, cap. 3; M5
Efeito alavancagem	Quedas de preço são seguidas de aumento da volatilidade	C1, cap. 4
Volume e volatilidade	O volume negociado é correlacionado com a volatilidade	M4
Assimetria entre escalas de tempo	Volatilidades medidas em escalas longas preveem as de escalas curtas melhor que o contrário	M5

C1: este caderno; M*n*: Módulo *n* do curso.

Todo modelo de mercado que se pretenda realista precisa reproduzir esses fatos. O passeio aleatório gaussiano não reproduz quase nenhum, como mostra a prática a seguir.

4.3 Prática principal: o Ibovespa contra uma amostra gaussiana

A prática principal do Módulo 1 compara os retornos diários do Ibovespa entre 2000 e 2025 com uma amostra sorteada de uma normal com a mesma média, o mesmo desvio-padrão e o mesmo número de dias. Se o modelo gaussiano fosse adequado, as duas séries seriam indistinguíveis. A Listagem 4.3 começa pelos números.

Listagem 4.3 O Ibovespa e uma amostra normal de mesma média e desvio.

```
# A referência: uma amostra normal com a média e o desvio do Ibovespa
rng = np.random.default_rng(seed=42)
gauss = pd.Series(rng.normal(r_ibov.mean(), r_ibov.std(), len(r_ibov)),
                  index=r_ibov.index)


def resumo(r):
    s = r.std()
    return {"desvio_%": 100 * s, "assimetria": stats.skew(r),
            "curtose": stats.kurtosis(r), "pior_%": 100 * r.min(),
            "melhor_%": 100 * r.max(), "dias>4dp": (abs(r) > 4 * s).sum()}


tabela = pd.DataFrame({"Ibovespa": resumo(r_ibov), "Normal": resumo(gauss)})
print(tabela.round(2))
```

```
            Ibovespa  Normal
desvio_%        1.70    1.71
assimetria     -0.35   -0.01
curtose         7.00    0.06
pior_%        -15.99   -7.44
melhor_%       13.68    5.92
dias>4dp       34.00    1.00
```

O desvio-padrão é o mesmo, por construção, mas tudo o mais difere. A pior queda do Ibovespa (-16%) e a maior alta ($+13{,}7\%$) são mais que o dobro das da amostra normal. Houve 34 dias com variações de mais de 4 desvios-padrão; a amostra normal teve um. A Figura 4.1 mostra as diferenças em quatro tipos de gráfico.

Listagem 4.4 Os quatro gráficos da prática principal.

```
fig, eixos = plt.subplots(3, 2, figsize=(6.4, 7.0))
(a, b), (c, d), (e, f) = eixos

for ax, serie, titulo in [(a, r_ibov, "Ibovespa"), (b, gauss, "Normal")]:
    ax.plot(serie.index, serie, linewidth=0.3)
    ax.set_ylim(-0.17, 0.17)
    ax.set_title(f"{titulo}: retornos diários")

bins = np.linspace(-0.17, 0.17, 80)
c.hist(r_ibov, bins=bins, alpha=0.6, label="Ibovespa")
c.hist(gauss, bins=bins, alpha=0.6, label="Normal")
c.set_yscale("log")
c.set_title("Histogramas (escala log)")
c.legend()

d.plot(np.sort(gauss), np.sort(r_ibov), ".", markersize=2)
d.plot([-0.17, 0.17], [-0.17, 0.17], color="C1", linewidth=0.8)
d.set_title("Quantis: Ibovespa contra normal")
```

```
19
20  k = np.arange(1, 51)
21  for ax, f_serie, titulo in [(e, lambda s: s, "Autocorrelação dos retornos"),
22                              (f, abs, "Autocorrelação dos absolutos")]:
23      ax.plot(k, acf(f_serie(r_ibov), nlags=50)[1:], label="Ibovespa")
24      ax.plot(k, acf(f_serie(gauss), nlags=50)[1:], label="Normal")
25      ax.axhline(0, color="black", linewidth=0.6)
26      ax.set_title(titulo)
27      ax.set_xlabel("defasagem")
28  e.legend()
29  fig.tight_layout()
30  fig.savefig("figuras/fig_04_01_pratica.pdf")
```

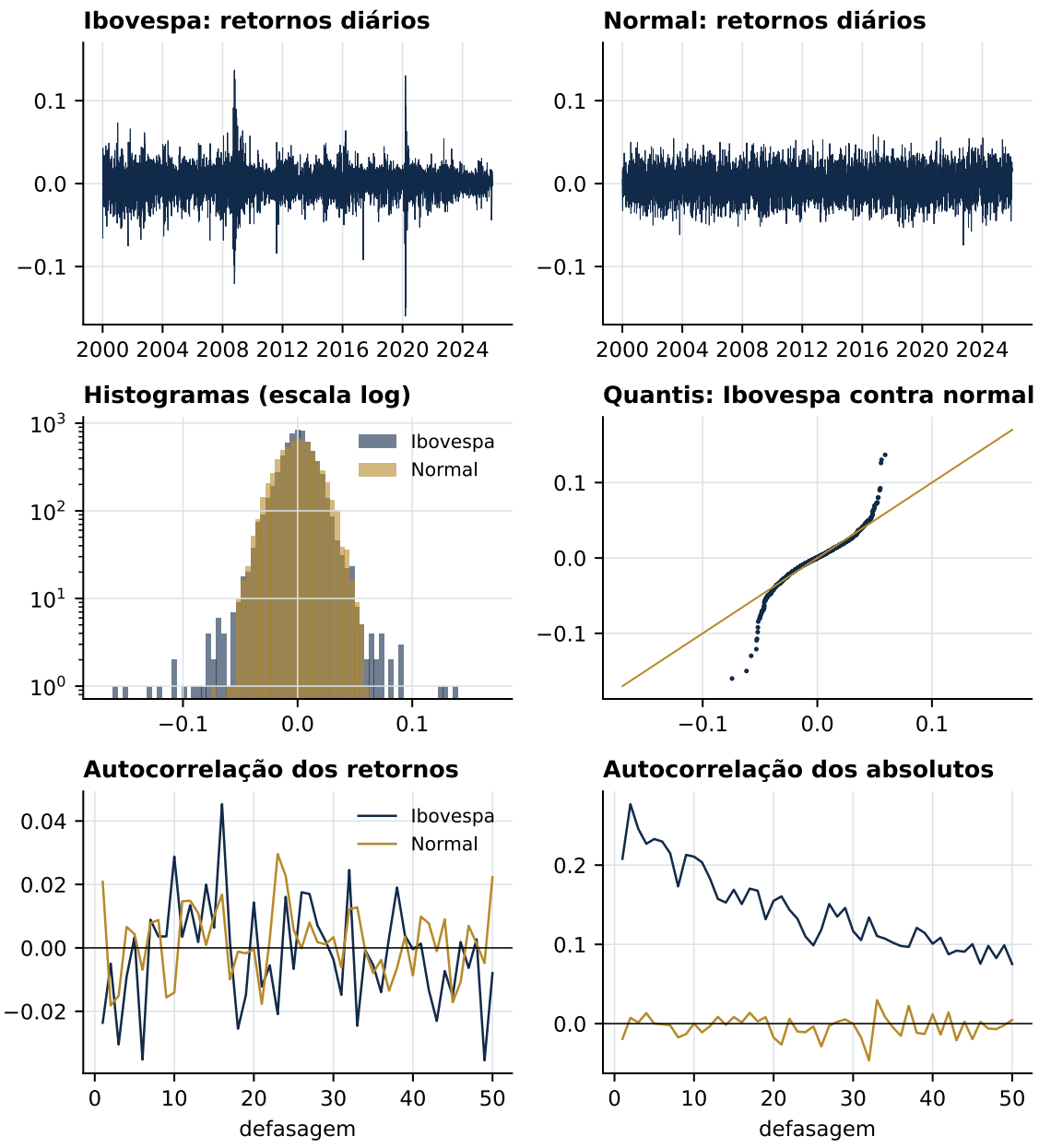


Figura 4.1 O Ibovespa (2000–2025) contra uma amostra normal de mesma média e desvio-padrão: séries temporais, histogramas em escala logarítmica, gráfico quantil-quantil e autocorrelações dos retornos e dos retornos absolutos.

Cada painel da Figura 4.1 mostra um fato estilizado:

- **Séries temporais** (no alto): no Ibovespa, os retornos grandes se concentram em períodos, como 2008 e 2020; na série normal, se espalham uniformemente. É o agrupamento de volatilidade.
- **Histogramas** (no meio, à esquerda): na escala logarítmica, o Ibovespa tem retornos muito além do alcance da normal, nas duas caudas. São as caudas pesadas.
- **Quantis** (no meio, à direita): no centro, os pontos seguem a reta; nas pontas, os quantis do Ibovespa são muito maiores que os da normal.
- **Autocorrelações** (embaixo): para os retornos, as duas séries ficam perto de zero; para os retornos absolutos, só o Ibovespa tem autocorrelação, e ela decai devagar.

Armadilha comum

Comparar os dados com uma única amostra simulada e tirar conclusões de diferenças pequenas. Uma amostra normal também varia de um sorteio para outro. As diferenças da Figura 4.1 são grandes demais para isso, mas, quando a diferença for sutil, repita a simulação com várias sementes e veja se o resultado se mantém (Exercício 4.2).

4.4 Gaussianidade agregacional

Se os retornos diários têm caudas pesadas, o que acontece com os retornos semanais, mensais ou anuais? Somar retornos de muitos dias é exatamente a situação do teorema central do limite. A Listagem 4.5 calcula a curtose em excesso dos retornos somados em blocos de h pregões, sem sobreposição.

Listagem 4.5 Curtose em excesso por horizonte.

```
def curtose_por_horizonte(r, horizontes):
    """Curtose em excesso dos retornos somados em blocos de h pregões."""
    x = np.asarray(r)
    resultado = {}
    for h in horizontes:
        n = len(x) // h
        resultado[h] = stats.kurtosis(x[:n * h].reshape(n, h).sum(axis=1))
    return pd.Series(resultado)


horizontes = [1, 5, 21, 63, 252]      # dia, semana, mês, trimestre, ano
tabela = pd.DataFrame({
    "S&P 500": curtose_por_horizonte(r_sp, horizontes),
    "Ibovespa": curtose_por_horizonte(r_ibov, horizontes)})
tabela.index.name = "pregões"
print(tabela.round(2))
```

```
         S&P 500  Ibovespa
pregões
1          25.28      7.00
5           5.28      8.24
21          5.56      5.66
63          2.64      1.38
252         1.17      0.48
```

A curtose cai à medida que o horizonte cresce: no S&P 500, de 25 nos retornos diários para pouco mais de 1 nos anuais. É a *gaussianidade agregacional*: em horizontes longos, a distribuição se aproxima da normal. Mas a aproximação é lenta, e não monotônica no Ibovespa, em que a curtose semanal é maior que a diária. Se os retornos fossem independentes, a curtose cairia aproximadamente como $1/h$, do mesmo jeito que no teorema central do limite da Listagem 4.2: de 25 para cerca de 5 em uma semana e para cerca de 1 em um mês. No S&P 500, ela ainda é 5,6 num mês. A dependência entre os dias, a mesma do agrupamento de volatilidade, atrasa a convergência.

Nota

Os horizontes longos têm poucos blocos: são 75 anos completos no S&P 500 e apenas 25 no Ibovespa. A curtose estimada com tão poucas observações é muito imprecisa, e os valores das últimas linhas da tabela devem ser lidos como indicação, não como medida.

4.5 Assimetrias: perdas e alavancagem

Os mercados não tratam altas e quedas do mesmo modo. A primeira assimetria está nos extremos: quedas grandes são mais numerosas do que altas grandes.

Listagem 4.6 Quedas e altas de mais de 3 desvios-padrão.

```
# Assimetria entre ganhos e perdas: quedas extremas são mais numerosas
for nome, r in [("S&P 500", r_sp), ("Ibovespa", r_ibov)]:
    s = r.std()
    print(f"{nome:<9} quedas > 3 dp: {(r < -3 * s).sum():3d}    "
          f"altas > 3 dp: {(r > 3 * s).sum():3d}    "
          f"assimetria: {stats.skew(r):+.2f}")
```

```
S&P 500   quedas > 3 dp: 145   altas > 3 dp: 124   assimetria: -0.95
Ibovespa  quedas > 3 dp:  42   altas > 3 dp:  30   assimetria: -0.35
```

No S&P 500, houve 145 quedas de mais de 3 desvios-padrão e 124 altas; no Ibovespa, 42 quedas e 30 altas. A assimetria negativa das duas séries resume isso num número.

A segunda assimetria está na dinâmica, e se chama *efeito alavancagem*: uma queda hoje é seguida de mais volatilidade nos dias seguintes, mas a volatilidade de hoje não diz muito sobre a direção dos próximos retornos.

Listagem 4.7 O efeito alavancagem: correlações num sentido e no outro.

```
# Efeito alavancagem: o retorno de hoje e a volatilidade dos dias seguintes
for nome, r in [("S&P 500", r_sp), ("Ibovespa", r_ibov)]:
    adiante = [r.corr(r.abs().shift(-k)) for k in (1, 5, 20)]
    atras = [r.abs().corr(r.shift(-k)) for k in (1, 5, 20)]
    print(f"{nome:<9} corr(r hoje, |r| em 1, 5, 20 dias): "
          f"{np.round(adiante, 3)}")
    print(f"{'':<9} corr(|r| hoje, r em 1, 5, 20 dias): "
          f"{np.round(atras, 3)}")
```

```
S&P 500   corr(r hoje, |r| em 1, 5, 20 dias): [-0.108 -0.081 -0.049]
          corr(|r| hoje, r em 1, 5, 20 dias): [ 0.025 -0.017  0.018]
Ibovespa  corr(r hoje, |r| em 1, 5, 20 dias): [-0.096 -0.089 -0.051]
          corr(|r| hoje, r em 1, 5, 20 dias): [ 0.027 -0.014  0.027]
```

A correlação entre o retorno de hoje e o tamanho do movimento de amanhã é negativa (cerca de $-0{,}1$ nos dois índices) e ainda é visível depois de 20 pregões. No sentido contrário, entre o tamanho do movimento de hoje e o retorno futuro, as correlações ficam perto de zero. O nome vem de uma das explicações propostas: quando o preço das ações cai, a alavancagem financeira das empresas aumenta, e com ela o risco.

Contexto de mercado

O efeito alavancagem tem uma consequência prática para a gestão de risco: a volatilidade tende a subir justamente depois das quedas, quando as carteiras já perderam valor. Os modelos de risco do Módulo 9 precisam levar isso em conta.

Resumo do capítulo

- A normal é a referência por causa do teorema central do limite, que exige choques independentes e de variância finita.
- Fatos estilizados: regularidades estatísticas presentes em muitos ativos, mercados e períodos (CONT, 2001).
- O Ibovespa contra uma normal de mesmo desvio: 34 dias além de 4 desvios-padrão contra 1; caudas pesadas, agrupamento de volatilidade e memória nos absolutos.
- Gaussianidade agregacional: a curtose cai com o horizonte, mas bem mais devagar do que se os retornos fossem independentes.
- Assimetria ganho-perda e efeito alavancagem: quedas extremas são mais numerosas e são seguidas de mais volatilidade.

Exercícios

Essencial — fixação

Exercício 4.1 Refaça a Figura 4.1 para o S&P 500 (1950–2025). Que painel mostra a maior diferença em relação ao Ibovespa?

Exercício 4.2 Repita a Listagem 4.3 com 20 sementes diferentes e anote o número de dias além de 4 desvios-padrão em cada amostra normal. O maior valor chega perto dos 34 do Ibovespa?

Aplicado — o mesmo método num problema novo

Exercício 4.3 Calcule a curtose por horizonte da Listagem 4.5 para uma versão *embaralhada* dos retornos do S&P 500. Compare com a dos dados originais e explique a diferença.

Exercício 4.4 Repita a Listagem 4.7 separadamente para os períodos 1950–1989 e 1990–2025 do S&P 500. O efeito alavancagem é estável no tempo?

Desafio — extensão aberta

Exercício 4.5 Escolha um ativo diferente (uma ação, uma moeda, um criptoativo) e verifique, com as ferramentas deste caderno, quantos dos fatos estilizados da Tabela 4.1 você consegue observar. Algum deles não aparece? Por quê?

CAPÍTULO 5

Críticas e diálogo com a econometria

◎ Objetivos do capítulo

- Conhecer as principais críticas feitas à Econofísica.
- Entender a resposta da econometria financeira ao agrupamento de volatilidade.
- Situar o curso entre as duas abordagens.

5.1 As críticas

O crescimento da Econofísica veio acompanhado de críticas, algumas vindas de economistas que simpatizavam com a área. Um artigo influente de Gallegati, Keen, Lux e Ormerod, com o título sugestivo de "Tendências preocupantes na Econofísica", reuniu quatro delas (GALLEGATI *et al.*, 2006):

1. **Desconhecimento da literatura econômica.** Trabalhos de Econofísica às vezes apresentam como novos resultados que a economia ou a econometria já conheciam.
2. **Pouco rigor estatístico.** Leis de potência, por exemplo, foram muitas vezes "confirmadas" por uma reta traçada a olho num gráfico log-log, sem teste nem comparação com alternativas.
3. **Crença em leis universais.** A expectativa de encontrar nos mercados leis tão universais quanto as da física nem sempre é sustentada pelos dados.
4. **Modelos inadequados.** Alguns modelos importam da física hipóteses que não fazem sentido econômico, como a conservação da quantidade total de dinheiro numa troca.

Essas críticas não invalidam a área, mas definem o padrão de qualidade que este curso procura seguir. O Módulo 4, por exemplo, aplica a leis de potência o protocolo estatístico de Clauset, Shalizi e Newman, justamente para não cair no segundo problema.

5.2 A resposta da econometria ao agrupamento de volatilidade

O agrupamento de volatilidade não é uma descoberta exclusiva da Econofísica. Em 1982, Robert Engle propôs o modelo ARCH (*autoregressive conditional heteroskedasticity*), em que a variância do retorno de hoje depende dos retornos passados (ENGLE, 1982). A generalização GARCH, de Tim Bollerslev, virou o modelo padrão de volatilidade da econometria financeira (BOLLERSLEV, 1986), e o trabalho de Engle foi reconhecido com o Prêmio Nobel de Economia de 2003.

O modelo ARCH(1) é simples de simular. O retorno de cada dia é $r_t = \sigma_t z_t$, com z_t normal padrão, e a variância depende do retorno do dia anterior:

$$\sigma_t^2 = \omega + \alpha\, r_{t-1}^2 .$$

Um retorno grande ontem aumenta a variância de hoje, o que torna mais provável outro retorno grande. A Listagem 5.1 simula o modelo e o compara com uma normal independente de mesma volatilidade média.

Listagem 5.1 Simulação de um modelo ARCH(1).

```
import matplotlib.pyplot as plt
import numpy as np
from scipy import stats
from statsmodels.tsa.stattools import acf

# ARCH(1): a variância de hoje depende do choque de ontem
rng = np.random.default_rng(seed=42)
n, omega, alfa = 6000, 0.00008, 0.5
z = rng.normal(size=n)
r = np.zeros(n)
for t in range(1, n):
    variancia = omega + alfa * r[t - 1] ** 2
    r[t] = np.sqrt(variancia) * z[t]
iid = rng.normal(0, r.std(), size=n)        # mesma volatilidade média

for nome, x in [("ARCH(1)", r), ("normal iid", iid)]:
    print(f"{nome:<10} curtose {stats.kurtosis(x):5.2f}   "
          f"acf|r| (1, 2, 3): {np.round(acf(np.abs(x), nlags=3)[1:], 3)}")
```

```
ARCH(1)    curtose  3.41   acf|r| (1, 2, 3): [0.398 0.19  0.097]
normal iid curtose  0.02   acf|r| (1, 2, 3): [-0.022 -0.003  0.018]
```

Listagem 5.2 Normal independente e ARCH(1), lado a lado.

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(6.4, 3.8), sharex=True,
                               sharey=True)
ax1.plot(iid, linewidth=0.3)
ax1.set_title("Normal independente")
ax2.plot(r, linewidth=0.3, color="C3")
ax2.set_title("ARCH(1): a volatilidade depende do passado")
ax2.set_xlabel("dia simulado")
fig.savefig("figuras/fig_05_01_arch.pdf")
```

Mesmo sendo construído com choques normais, o ARCH(1) produz caudas pesadas (curtose em excesso de 3,4) e autocorrelação nos retornos absolutos (0,40 na primeira defasagem): dois dos fatos estilizados do Capítulo 4, gerados por um único mecanismo. A memória

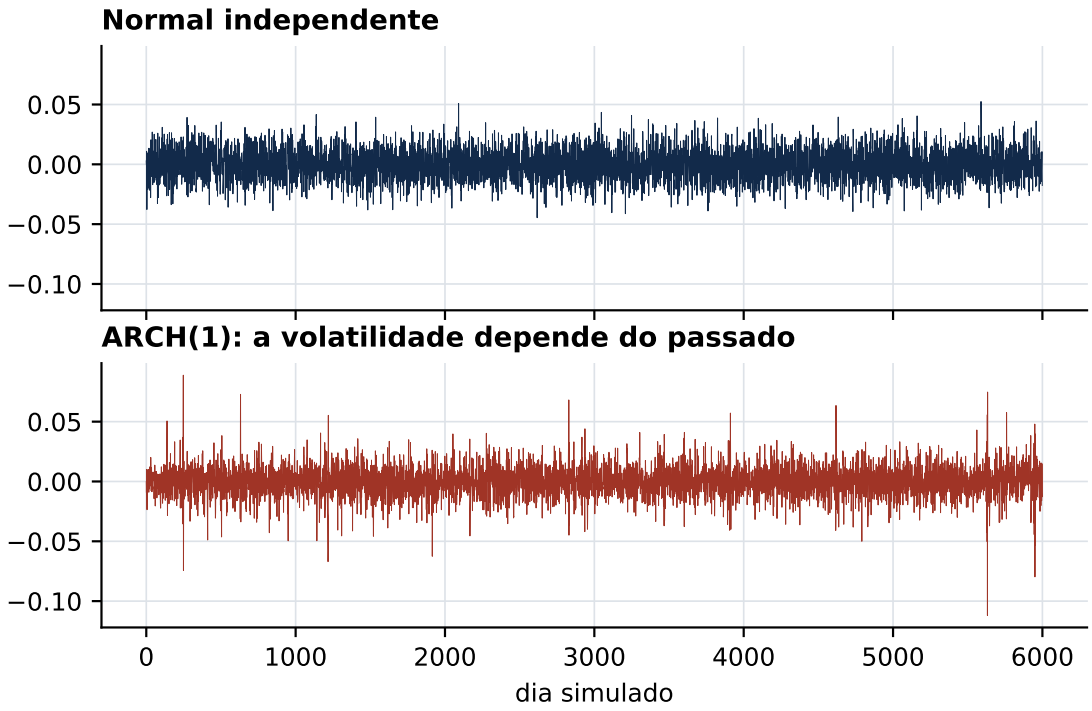


Figura 5.1 Retornos simulados de uma normal independente (acima) e de um modelo ARCH(1) com a mesma volatilidade média (abaixo). No ARCH, os retornos grandes aparecem em surtos.

da volatilidade, porém, decai depressa, em poucos dias, enquanto a dos dados reais dura meses (Figura 3.1). Modelos econométricos mais elaborados e os modelos da Econofísica dos Módulos 5 e 8 procuram reproduzir essa memória longa.

Contexto de mercado

O ARCH e as medidas de escala da Econofísica olham o mesmo fenômeno de ângulos diferentes. O primeiro ajusta parâmetros a uma série e serve para prever a volatilidade de amanhã; as segundas medem leis de escala e perguntam que mecanismo coletivo as produz. Um analista de risco se beneficia das duas visões.

5.3 Como este curso se posiciona

Este curso trata a Econofísica como um conjunto de ferramentas e de perguntas que complementam as finanças e a econometria, e não como uma escola rival. Na prática, isso se traduz em três compromissos, que você já viu em ação neste caderno:

- **comparar** cada resultado com a referência gaussiana e, quando houver, com o modelo clássico correspondente;
- **testar** as afirmações com estatística adequada, e não só com gráficos;
- **reconhecer** o que os dados não sustentam, como uma autocorrelação estatisticamente significativa, mas pequena demais para ter importância econômica.

Resumo do capítulo

- Críticas à Econofísica (Gallegati *et al.*, 2006): desconhecimento da literatura econômica, pouco rigor estatístico, crença em leis universais e modelos inadequados.
- A econometria modela o agrupamento de volatilidade com ARCH (Engle, 1982) e GARCH (Bollerslev, 1986).
- O ARCH(1) gera caudas pesadas e autocorrelação nos absolutos a partir de choques normais, mas com memória curta.
- O curso combina as duas abordagens: compara, testa e reconhece os limites dos dados.

Exercícios

Essencial — fixação

Exercício 5.1 Na Listagem 5.1, use $\alpha = 0{,}2$ e $\alpha = 0{,}8$. O que acontece com a curtose e com a autocorrelação dos absolutos?

Aplicado — o mesmo método num problema novo

Exercício 5.2 Calcule o correlograma dos retornos absolutos do ARCH(1) simulado até 100 defasagens e compare-o com o do Ibovespa (Figura 3.1). Em quantos dias a autocorrelação do modelo entra na faixa de 95%?

Desafio — extensão aberta

Exercício 5.3 Leia o artigo de Gallegati *et al.* (2006) e escolha uma das quatro críticas. Procure, na literatura de Econofísica posterior a 2006, um trabalho que tenha respondido a ela, e resuma a resposta em um parágrafo.

Referências do Caderno 1

As referências a seguir reúnem as fontes citadas neste caderno, organizadas por tema. As datas de acesso aos recursos eletrônicos referem-se a 10 de outubro de 2026.

História: do passeio aleatório às caudas pesadas

BACHELIER, L. Théorie de la spéculation. **Annales scientifiques de l'École Normale Supérieure**, série 3, v. 17, p. 21–86, 1900.

A tese que inaugura o passeio aleatório em finanças (Capítulo 2).

BLACK, F.; SCHOLES, M. The pricing of options and corporate liabilities. **Journal of Political Economy**, v. 81, n. 3, p. 637–654, 1973.

FAMA, E. F. The behavior of stock-market prices. **The Journal of Business**, v. 38, n. 1, p. 34–105, 1965.

MANDELBROT, B. The variation of certain speculative prices. **The Journal of Business**, v. 36, n. 4, p. 394–419, 1963.

A primeira crítica sistemática ao modelo gaussiano.

OSBORNE, M. F. M. Brownian motion in the stock market. **Operations Research**, v. 7, n. 2, p. 145–173, 1959.

Econofísica

MANTEGNA, R. N.; STANLEY, H. E. Scaling behaviour in the dynamics of an economic index. **Nature**, v. 376, n. 6535, p. 46–49, 1995.

MANTEGNA, R. N.; STANLEY, H. E. **An introduction to econophysics**: correlations and complexity in finance. Cambridge: Cambridge University Press, 1999.

Texto de referência da área; os Capítulos 1 a 4 cobrem os temas deste caderno.

GALLEGATI, M. *et al.* Worrying trends in econophysics. **Physica A**, v. 370, n. 1, p. 1–6, 2006.

As críticas discutidas no Capítulo 5.

Mercados eficientes

FAMA, E. F. Efficient capital markets: a review of theory and empirical work. **The Journal of Finance**, v. 25, n. 2, p. 383–417, 1970.

LO, A. W.; MacKINLAY, A. C. Stock market prices do not follow random walks: evidence from a simple specification test. **The Review of Financial Studies**, v. 1, n. 1, p. 41–66, 1988.

SAMUELSON, P. A. Proof that properly anticipated prices fluctuate randomly. **Industrial Management Review**, v. 6, n. 2, p. 41–49, 1965.

Fatos estilizados e volatilidade

BOLLERSLEV, T. Generalized autoregressive conditional heteroskedasticity. **Journal of Econometrics**, v. 31, n. 3, p. 307–327, 1986.

CONT, R. Empirical properties of asset returns: stylized facts and statistical issues. **Quantitative Finance**, v. 1, n. 2, p. 223–236, 2001.

O catálogo de fatos estilizados da Tabela 4.1; leitura obrigatória.

DING, Z.; GRANGER, C. W. J.; ENGLE, R. F. A long memory property of stock market returns and a new model. **Journal of Empirical Finance**, v. 1, n. 1, p. 83–106, 1993.

ENGLE, R. F. Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. **Econometrica**, v. 50, n. 4, p. 987–1007, 1982.

Estatística e software

LJUNG, G. M.; BOX, G. E. P. On a measure of lack of fit in time series models. **Biometrika**, v. 65, n. 2, p. 297–303, 1978.

SEABOLD, S.; PERKTOLD, J. Statsmodels: econometric and statistical modeling with Python. In: PROCEEDINGS OF THE 9TH PYTHON IN SCIENCE CONFERENCE (SciPy 2010). Austin, 2010. p. 92–96.

STATSMODELS DEVELOPERS. **statsmodels documentation**. Disponível em: `https://www.statsmodels.org/stable/`. Acesso em: 10 out. 2026.

AROUSSI, R. **yfinance**: download market data from Yahoo! Finance API. Disponível em: `https://github.com/ranaroussi/yfinance`. Acesso em: 10 out. 2026.

Biblioteca não oficial, não afiliada ao Yahoo; leia os termos de uso indicados na documentação.